Theory of Automata
Kleene's Theorem

Contents

- Unification
- Turning TGs into Regular Expressions
- Converting Regular Expressions into FAs
- Nondeterministic Finite Automata
- NFAs and Kleene’s Theorem

Unification

- We have learned three separate ways to define a language: (i) by **regular expression**, (ii) by **finite automaton**, and (iii) by **transition graph**.

- Now, we will present a theorem proved by Kleene in 1956, which says that **if a language can be defined by any one of these three ways, then it can also be defined by the other two**.

- In other words, Kleene proved that **all three of these methods of defining languages are *equivalent*.**

Theorem 6 - Kleene's Theorem

- Any language that can be defined by regular expression, or finite automaton, or transition graph can be defined by all three methods.

Kleene's Theorem

- This theorem is the most important and fundamental result in the theory of finite automata.
- We will take extreme care with its proofs. In particular, we will introduce four algorithms that enable us to construct the corresponding machines and expressions.
- Recall that
  - To prove A = B, we need to prove (i) $A \subset B$, and (ii) $B \subset A$.
  - To prove A = B = C, we need to prove (i) $A \subset B$, (ii) $B \subset C$, and (iii) $C \subset A$.

Kleene's Theorem

- Thus, to prove Kleene's theorem, we need to prove 3 parts:
- **Part 1:** Every language that can be defined by a finite automaton can also be defined by a transition graph.
- **Part 2:** Every language that can be defined by a transition graph can also be defined by a regular expression.
- **Part 3:** Every language that can be defined by a regular expression can also be defined by a finite automaton.

Proof of Part 1

- This is the easiest part.

- From previous lecture, we know that every finite automaton is itself already a transition graph. Therefore, any language that has been defined by a finite automaton has already been defined by a transition graph.

Turning TGs into Regular Expressions

Proof of Part 2: Turning TGs into Regular Expressions

- We prove this part by providing a **constructive algorithm**:
  - We present an algorithm that starts out with a transition graph and ends up with a regular expression that defines the same language.
  - To be acceptable as a method of proof, the algorithm we present will satisfy two criteria:
    (i) It works for every conceivable TG, and
    (ii) It guarantees to finish its job in a finite number of steps.
- Slides 8 - 27 below present the proof of part 2.

Step-1 Creating A Unique Start State

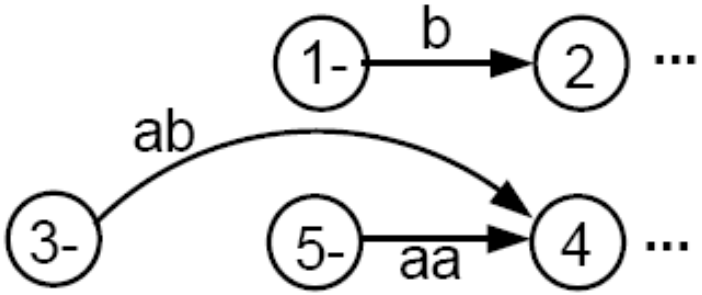
Consider an abstract transition graph T that may have many start states.

- We can simplify T so that it has **only one unique start state that has no incoming edges**.

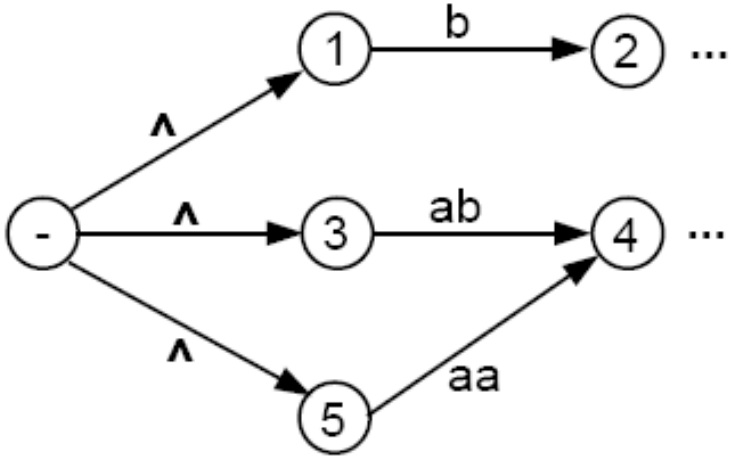
- We do this by introducing a new start state that we label with the minus sign, and that we connect to all the previous start states by edges labeled with Λ. We then drop the minus signs from the previous start states.

- If a word w used to be accepted by starting at one of the previous start states, then it can now be accepted by starting at the new unique start state.
- The following figure illustrates this idea.

- Consider a fragment of T:



- The above fragment of T can be replaced by

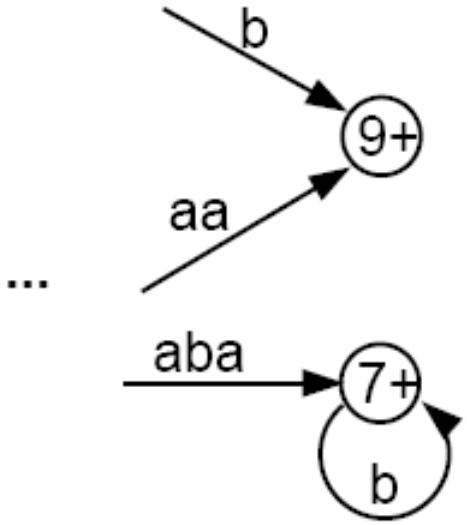


Step 2 - Creating a Unique Final State

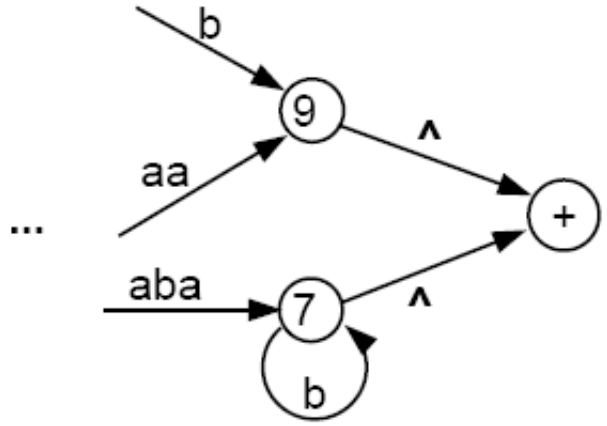
- Let us make another simplification in T so that it has a **unique, un-exitable final state**, without changing the language it accepts.

- If T had no final state, then it accepts no strings at all and has no language. So, we need to produce no regular expression other than the null, or empty, expression Φ(see page 36 of the text).

- If T has several final states, we can introduce a new unique final state labeled with a plus sign. We then draw new edges from all the former final states to the new one, dropping the old plus signs, and labeling each new edge with the null string.
- This process is depicted in the next slide.



becomes



- We shall require that the unique final state be a different state from the unique start state. If an old state used to have ±, then both signs are removed from the old state to newly created states, using the processes described above.

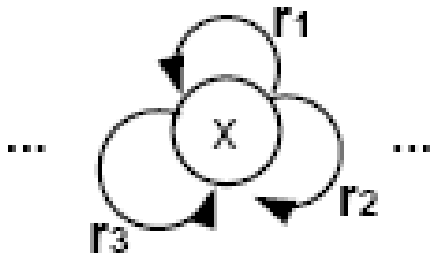
- It should be clear that the addition of these two new states does not affect the language that T accepts.

- The machine now has the following shape:

where there are no other - or + states.

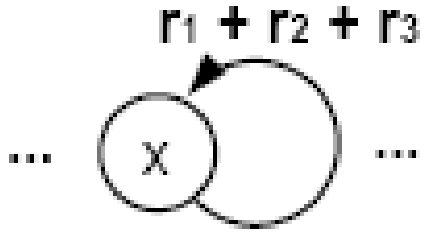
Step 3 -- Combining Edges

- If T has some internal state x (not the - or the + state) that has more than one loop circling back to itself:



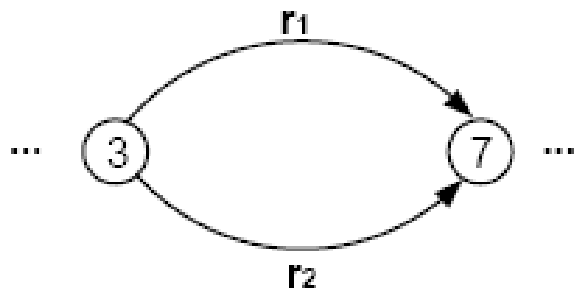
where r_1, r_2, and r_3 are all regular expressions or simple strings.

- We can replace the three loops by one loop labeled with a regular expression:

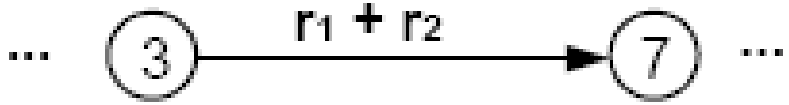


Combining Edges

- Similarly, if two states are connected by **more than one edge** going in the same direction:

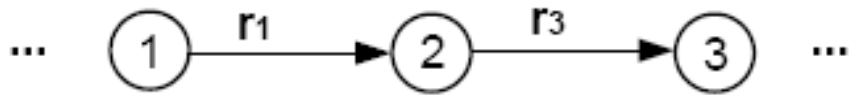


- We can replace this with a single edge labeled with a regular expression:

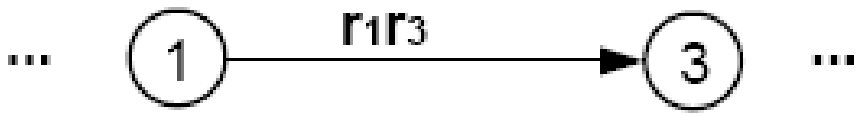


Bypass and State Elimination

- If T has 3 states in a row connected by edges labeled with regular expressions or simple strings, we can eliminate the middle state, as in the following examples:

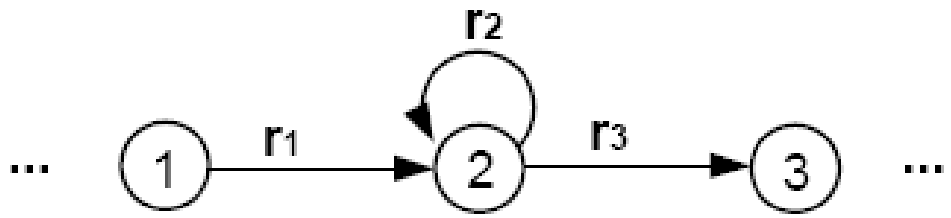


Can be replaced with

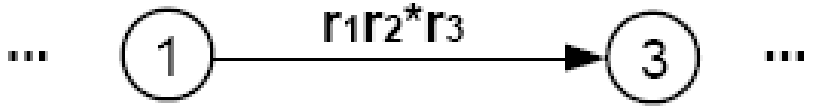


Bypass and State Elimination

- If the middle state has a loop, we can proceed as follows:



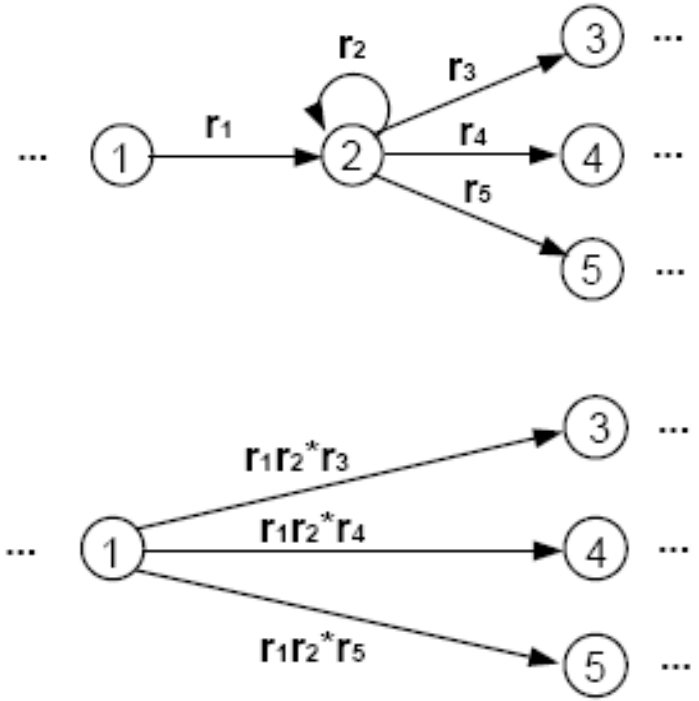
Can be replaced with



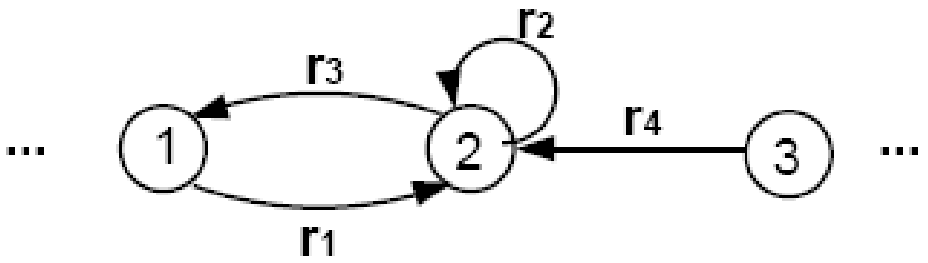
Bypass and State Elimination

- If the middle state is connected to more than one state, then the bypass and elimination process can be done as follows:

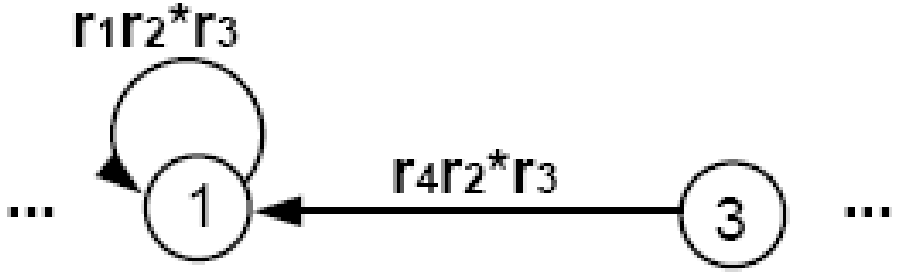
Can be replaced with



Special Cases

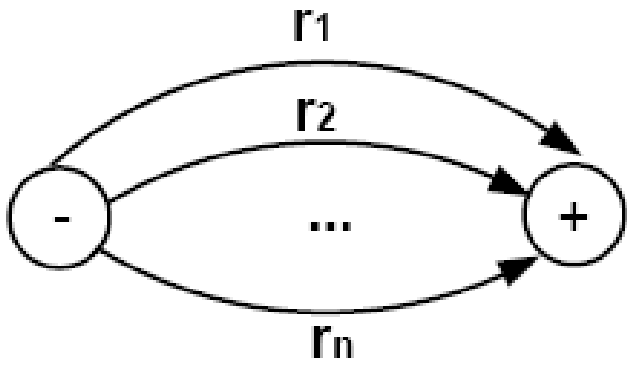


Can be replaced with



Combining Edges

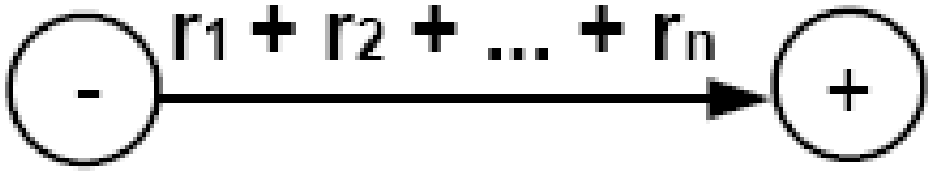
- We can repeat this bypass and elimination process again and again until we have eliminated all the states from T, except for the unique start state and the unique final state. What we come down to is a picture that looks like this:



with each edge labeled by a regular expression.

Combing Edges

- We can then combine the edges from the above picture one more to produce

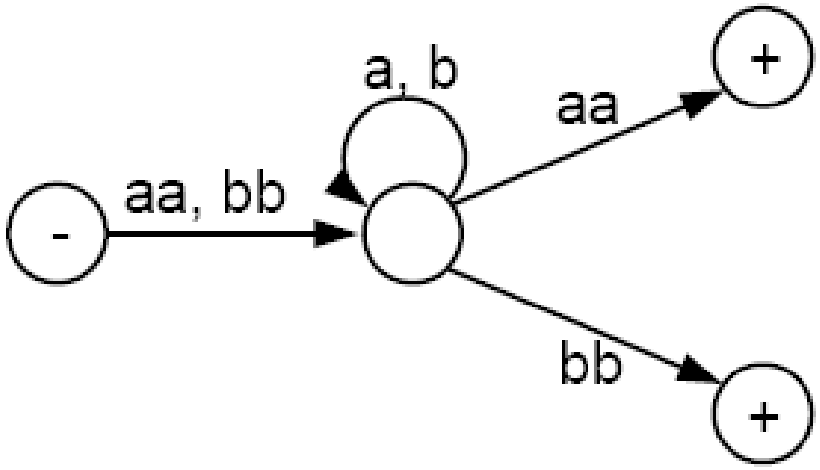


in which the resultant expression is the regular expression that defines the same language as *T* did originally.

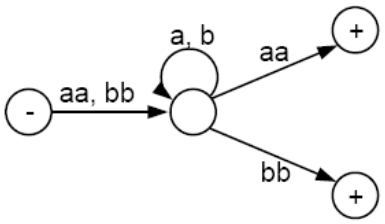
- Recall that all words accepted by *T* are paths through the picture of *T*. If we change the picture but preserve all paths and their labels, we must keep the language unchanged.

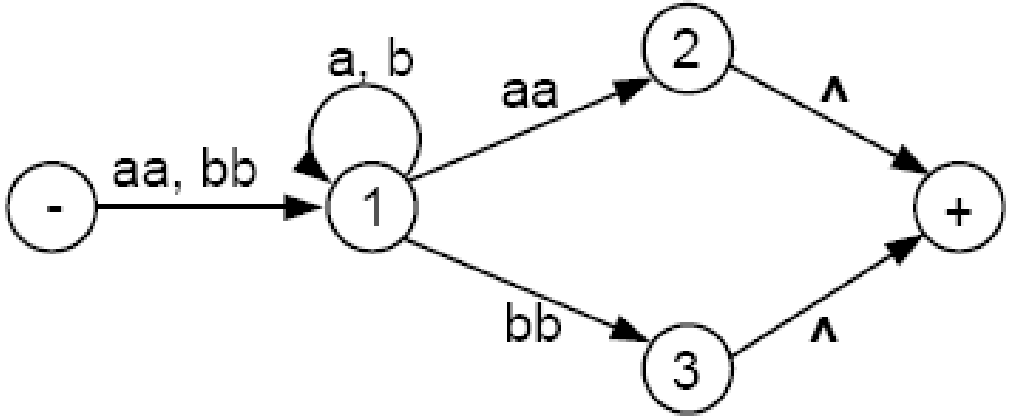
Example

- Consider the following TG that accepts all words that begin and end with double letters (having at least length 4):

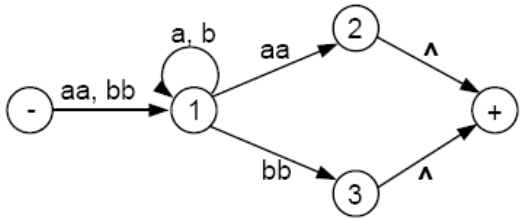


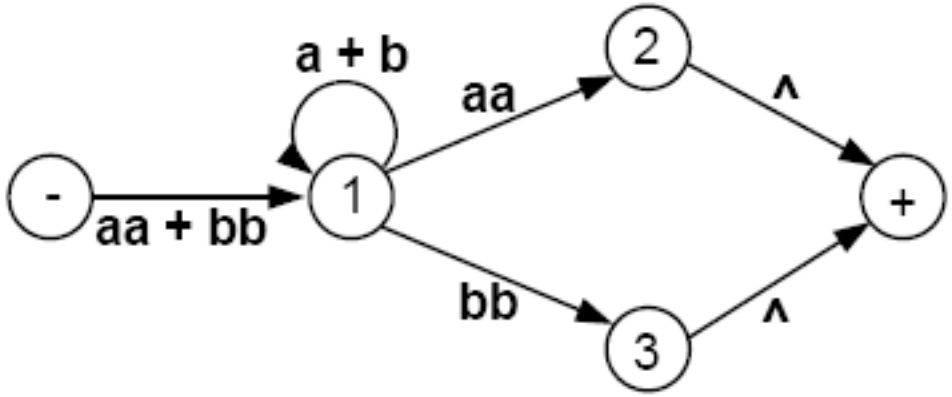
- This TG has only one start state with no incoming edges, but has two final states. So, we must introduce a new unique final state:



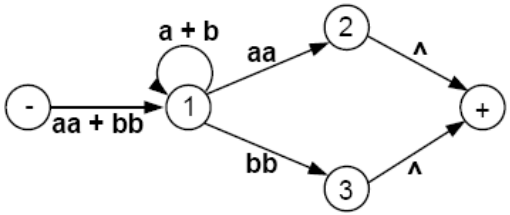


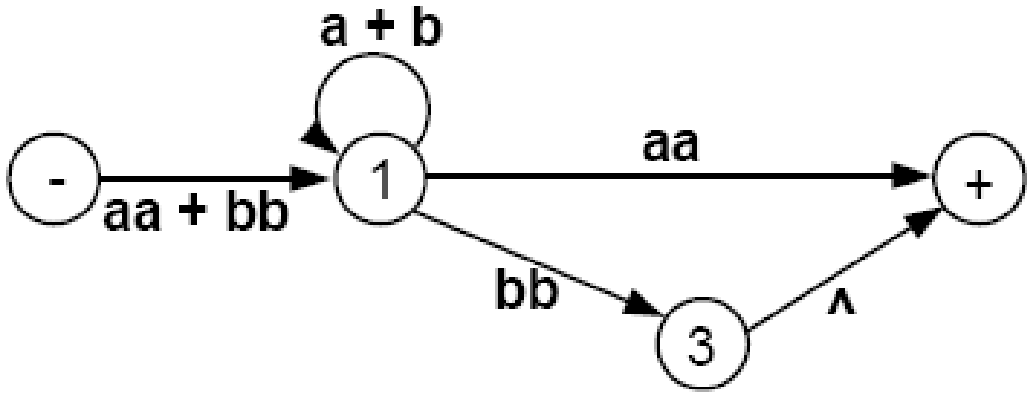
- Now we build regular expressions piece by piece:



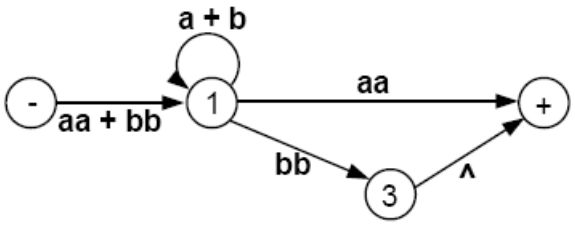


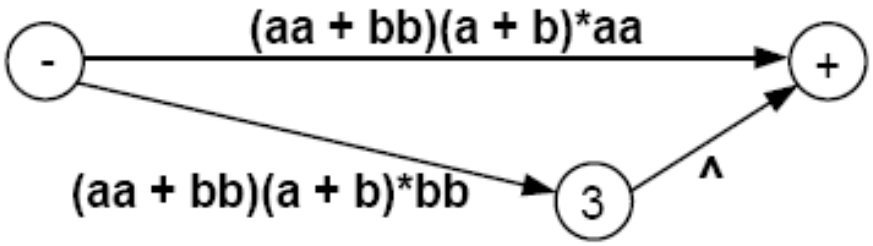
- Eliminate state 2:



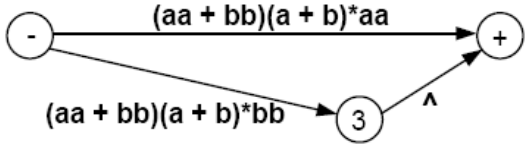


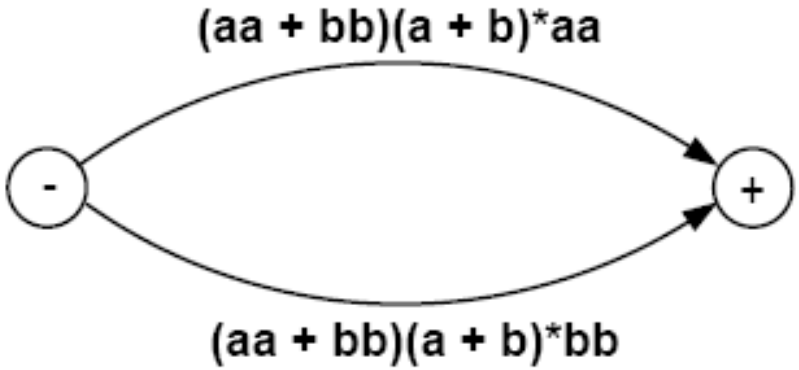
Eliminate state 1:





Eliminate state 3:





- Hence, this TG defines the same language as the regular expression

  (aa + bb)(a + b)*(aa) + (aa + bb)(a + b)*(bb)

- or equivalently

  (aa + bb)(a + b)*(aa + bb)

- If we eliminated the states in a different order, we could end up with a different-looking regular expression. But by the logic of the elimination process, these expressions would all have to represent the same language.

- We are now ready to present the constructive algorithm that proves that all TGs can be turned into regular expressions that define the exact same language.

Algorithm

- Step 1: Create a unique, unenterable minus state and a unique, unleaveable plus state.

- Step 2: One by one, in any order, bypass and eliminate all the non-minus or non-plus states in the TG. A state is bypassed by connecting each incoming edge with each outgoing edge. The label of each resultant edge is the concatenation of the label on the incoming edge with the label on the loop edge (if there is one) and the label on the outgoing edge.

- Step 3: When two states are joined by more than one edge going in the same direction, unify them by adding their labels.

- Step 4: Finally, when all that is left is one edge from - to +, the label on that edge is a regular expression that generates the same language as was recognized by the original TG.

Algorithm for *Finite Automaton to Regular Expression* Conversion 1a

Assume that we have a DFA or an NFA. Perform the following steps:

1. Create a new start state ***s***, and draw a new edge labeled with λ from ***s*** to the original start state.

2. Create a new final state ***f***, and draw new edges labeled with λ from all the original final states to ***f.***

3. For each pair of states ***i*** and ***j*** that have more than one edge from ***i*** to ***j***, replace all the edges from *i* to ***j*** by a single edge labeled with the regular expression formed by the sum of the labels on each of the edges from ***i*** to ***j***.

4. Construct a sequence of new machines by eliminating one state at a time until the only states remaining are ***s*** and the ***f.*** As each state is eliminated, a new machine is constructed from the previous machine as follows:

Eliminate State k

For convenience we'll let ***old(i,j)*** denote the label on ***edge (i, j)*** of the current machine. If there is no edge ***(i, j),*** then ***set old(i, j) = φ***. Now for each pair of edges ***(i, k)*** and ***(k, j),*** where $i \neq k$ ***and*** $j \neq k$, calculate a new edge label, ***new(z, y),*** as follows:

$$new(i, j) = old(i, j) + old(i, k)\ old(k, k)^* old(k, j).$$

For all other edges ***(i,j)*** where $i \neq k$ ***and*** $j \neq k$, set

$$new(i, j) = old(i, j).$$

The states of the new machine are those of the current machine with state ***k*** eliminated. The edges of the new machine are the ***edges (i, j)*** for which label ***new(i , j)*** has been calculated.

After eliminating all states except ***s*** and ***f***, we wind up with a two-state machine with the single ***edge (s, f)*** labeled with the regular expression ***new(s, f),*** which represents the regular language accepted by the original finite automaton.

Converting Regular Expressions into FAs

Proof of Part 3: Converting Regular Expressions into FAs

- We prove this part by **recursive definition** and **constructive algorithm** at the same time.
  - We know that every regular expression can be built up from the letters of the alphabet ∑ and Λ by repeated application of certain rules: (i) addition, (ii) concatenation, and (iii) closure.
  - We will show that as we are building up a regular expression, we could at the same time building up an FA that accepts the same language.
- Slides 3 - 30 below show the proof of part 3.

- Before we proceed, let's have a quick review of the **formal definition of regular expressions**.
- The set of **regular expressions** is defined by the following rules:
  - Rule 1: Every letter of the alphabet ∑ can be made into a regular expression by writing it in **boldface: Λ** itself is a regular expression.
  - Rule 2: If r_1 and r_2 are regular expressions, then so are:
    - (r_1)
    - $r_1 r_2$
    - $r_1 + r_2$
    - r_1^*
  - Rule 3: Nothing else is a regular expression.

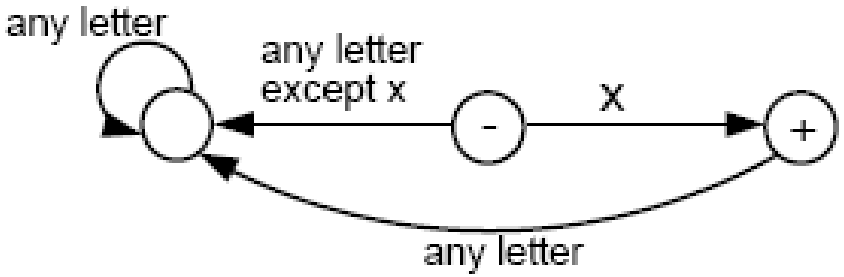
We now present proof of part 3 recursively.

Rule 1

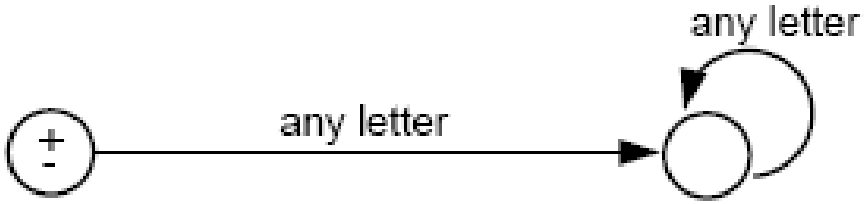
- There is an FA that accepts any particular letter of the alphabet.
- There is an FA that accepts only the word Λ.

Proof of rule 1

- If letter x is in ∑, then the following FA accepts only the word **x**.



- The following FA accepts only **λ**:



Rule 2

- If there is an FA called FA_1 that accepts the language defined by the regular expression r_1, and there is an FA called FA_2 that accepts the language defined by the regular expression r_2, then there is an FA that we shall call FA_3 that accepts the language defined by the regular expression $(r_1 + r_2)$.

Proof of Rule 2

- We shall show that FA_3 exists by presenting an algorithm showing how to construct FA_3.
- **Algorithm:**
  - Starting with two machines, FA_1 with states $x_1; x_2; x_3;\ldots$, and FA_2 with states $y_1; y_2; y_3; \ldots$, we construct a new machine FA_3 with states $z_1; z_2; z_3; \ldots$ where each z_i is of the form $x_{something}$ or $y_{something}$.
  - The combination state x_{start} or y_{start} is the start state of the new machine FA_3.
  - If either the *x* part or the *y* part is a final state, then the corresponding *z* is a final state.

Algorithm (cont.)

– To go from one state *z* to another by reading a letter from the input string, we observe what happens to the *x* part and what happens to the *y* part and go to the new state *z* accordingly. We could write this as a formula:

z_{new} after reading letter p = (x_{new} after reading letter p on FA_1) or (y_{new} after reading letter p on FA_2)

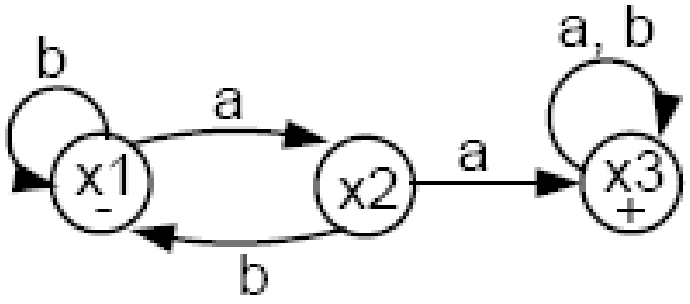
Remarks

- The new machine FA_3 constructed by the above algorithm will simultaneously keep track of where the input would be if it were running on FA_1 alone, and where the input would be if it were running on FA_2 alone.
- If a string traces through the new machine FA_3 and ends up at a final state, it means that it would also end at a final state either on machine FA_1 or on machine FA_2. Also, any string accepted by either FA_1 or FA_2 will be accepted by this FA_3. So, the language FA_3 accepts is the **union** of the languages accepted by FA_1 and FA_2, respectively.
- Note that since there are only finitely many states *x's* and finitely many states *y's*, there can be only finitely many possible states *z's*.
- Let us look at an example illustrating how the algorithm works.

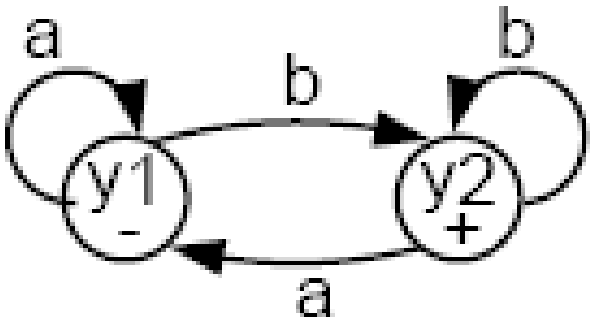
Example

- Consider the following two FAs:

FA1



FA2



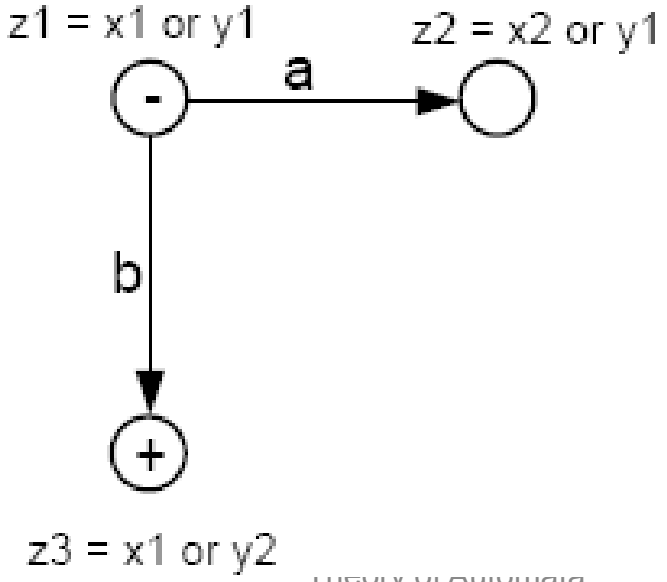
- FA_1 accepts all words with a double a in them.
- FA_2 accepts all words ending with b.
- Let's follow the algorithm to build FA_3 that accepts the union of the two languages.

Combining the FAs

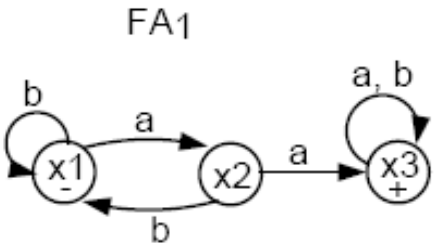
- The start (-) state of FA_3 is $z_1 = x_1$ or y_1.
- In z_1, if we read an ***a***, we go to x_2 (observing FA_1), or we go to y_1 (observing FA_2).

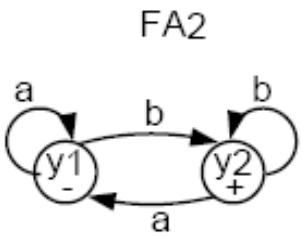
  Let $z_2 = x_2$ or y_1.
- In z_1, if we read a ***b***, we go to x_1 (observing FA_1), or to y_2 (observing FA_2).

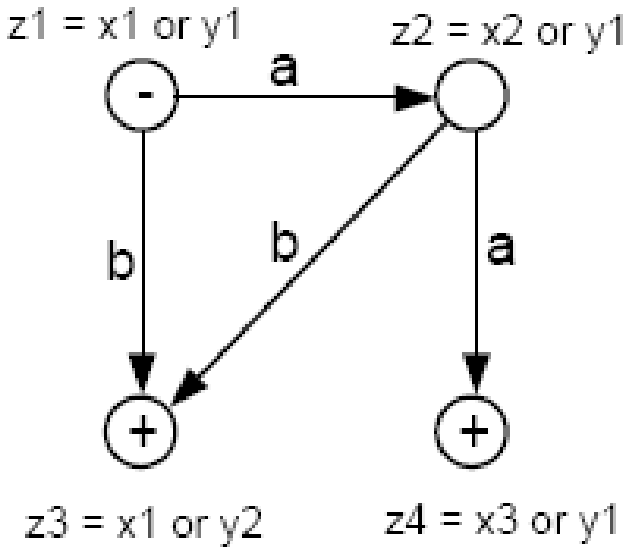
Let $z_3 = x_1$ or y_2. Note that z_3 must be a final state since y_2 is a final state.



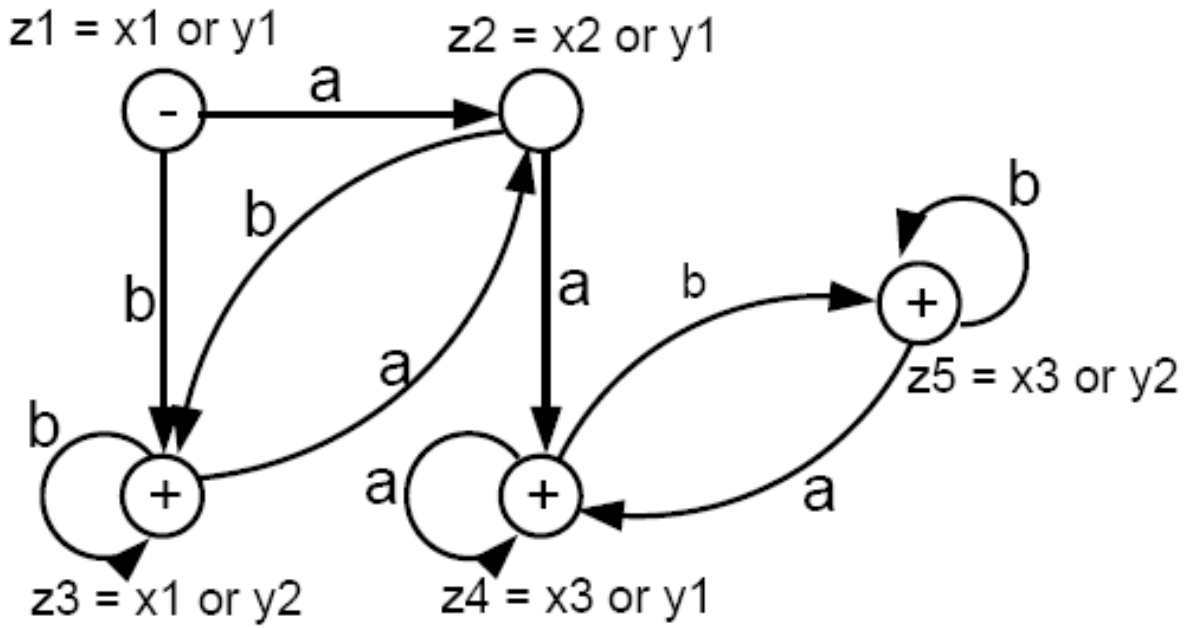
- In z_2, if we read an a, we go to x_3 or y_1. Let $z_4 = x_3$ or y_1. z_4 is a final state because x_3 is.
- In z_2, if we read a b, we go to x_1 or y_2, which is z_3.







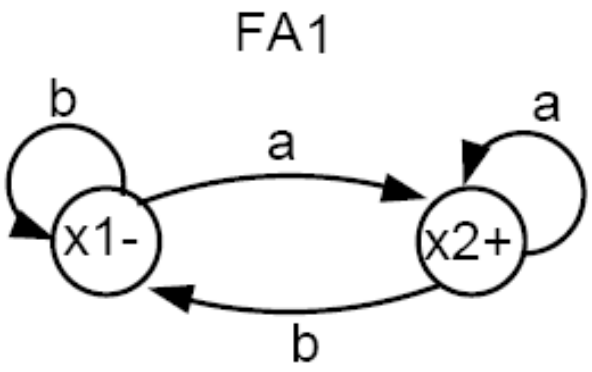
- In z_3, if we read an a, we go to x_2 or y_1, which is z_2.
- In z_3, if we read a b, we go to x_1 or y_2, which is z_3.
- In z_4, if we read an a, we go to x_3 or y_1, which is z_4. Hence, we have an a-loop at z_4.
- In z_4, if we read a b, we go to x_3 or y_2. Let $z_5 = x_3$ or y_2. Note that z_5 is a final state because x_3 (and y_2) are.
- In z_5, if we read an a, we go to x_3 or y_1, which is z_4.
- In z_5, if we read a b, we go to x_3 or y_2, which is z_5. Hence, there is a b-loop at z_5.
- The whole machine looks like the following:

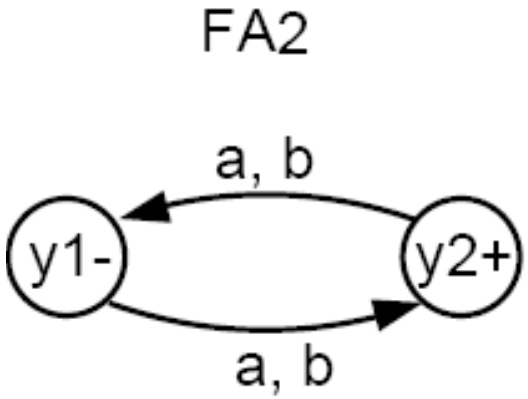


- This machine accepts all words that have a double *a* or that end with *b*.
- The labels $z_1 = x_1$ or y_1, $z_2 = x_2$ or y_1, etc. can be removed if you want.

Example

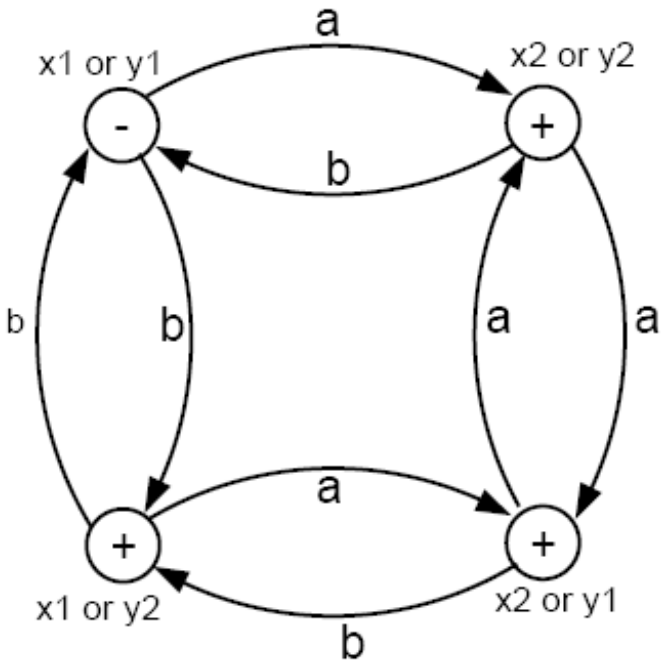
- Consider the following two FAs:





- FA_1 accepts all words that end in *a*.
- FA_2 accepts all words with an odd number of letters (odd length).
- Can you use the algorithm to build a machine FA3 that accepts all words that either have an odd number of letters or end in *a*?

- Using the algorithm, we can produce FA_3 that accepts all words that either have an odd number of letters or end in a, as follows:



- The only state that is not a + state is the - state. To get back to that start state, a word must have an even number of letters **and** end in *b*.

Rule 3

If there is an FA_1 that accepts the language defined by the regular expression r_1, and there is an FA_2 that accepts the language defined by the regular expression r_2, then there is an FA_3 that accepts the language defined by the (concatenation) regular expression $(r_1 r_2)$, i.e. the product language.

- We shall show that such an FA_3 exists by presenting an algorithm showing how to construct it from FA_1 and FA_2.

- The idea is to construct a machine that starts out like FA_1 and follows along it until it enters a final state at which time an option is reached. Either we continue along FA_1, waiting to reach another +, or else we switch over to the start state of FA_2 and begin circulating there.

Algorithm

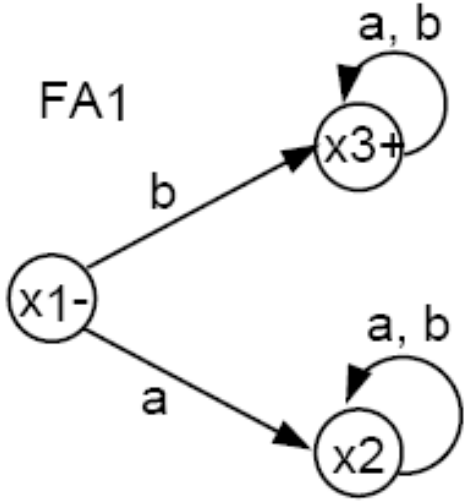
- First, create a state *z* for every state of FA_1 that we may go through before arriving at a final state.

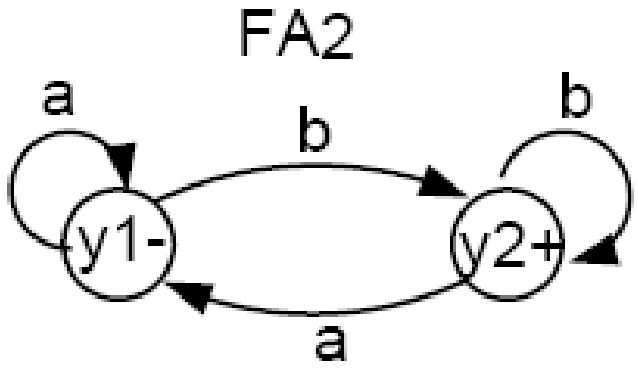
2. For each final state x_{final} of FA_1, add a state $z = x_{final}$ or y_1, where y_1 is the start state of FA_2.
3. From the states added in step 2, add states

$$z = \begin{cases} x_{something} \text{ indicating that we are still continuing on } FA_1 \\ \text{OR} \\ \text{a set of } y_{something} \text{ indicating that we are on } FA_2 \end{cases}$$

4. Label every state *z* that contains a final state from FA_2 as a final state.

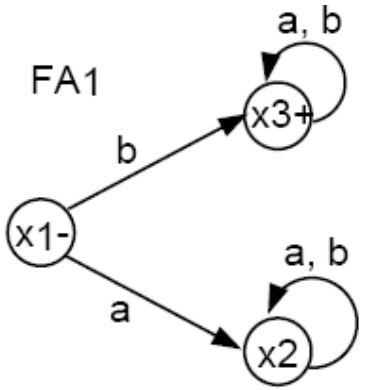
Example

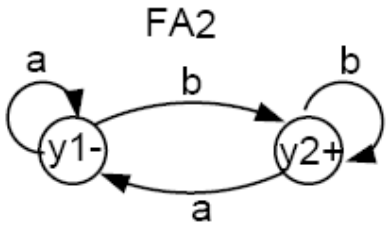


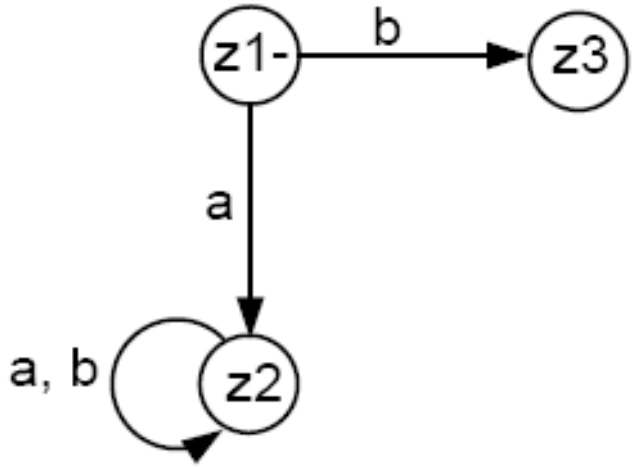


- FA_1 accepts all words that start with a *b*.
- FA_2 accepts all words that end with a *b*.
- We will use the above algorithm to construct FA_3 that accepts the product of the languages of FA_1 and FA_2, respectively. That is, FA_3 will accept all words that both start and end with the letter *b*.

- Initially, we must begin with $x_1 = z_1$.
- In z_1, if we read an a, we go to $x_2 = z_2$.
- In z_1, if we read a b, we go to x_3, a final state, which gives us the option to jump to y_1. Hence, we label $z_3 = x_3$ or y_1.
- From z_2 just like x_2, both an a or a b take us back to z_2, i.e., we have a loop here.





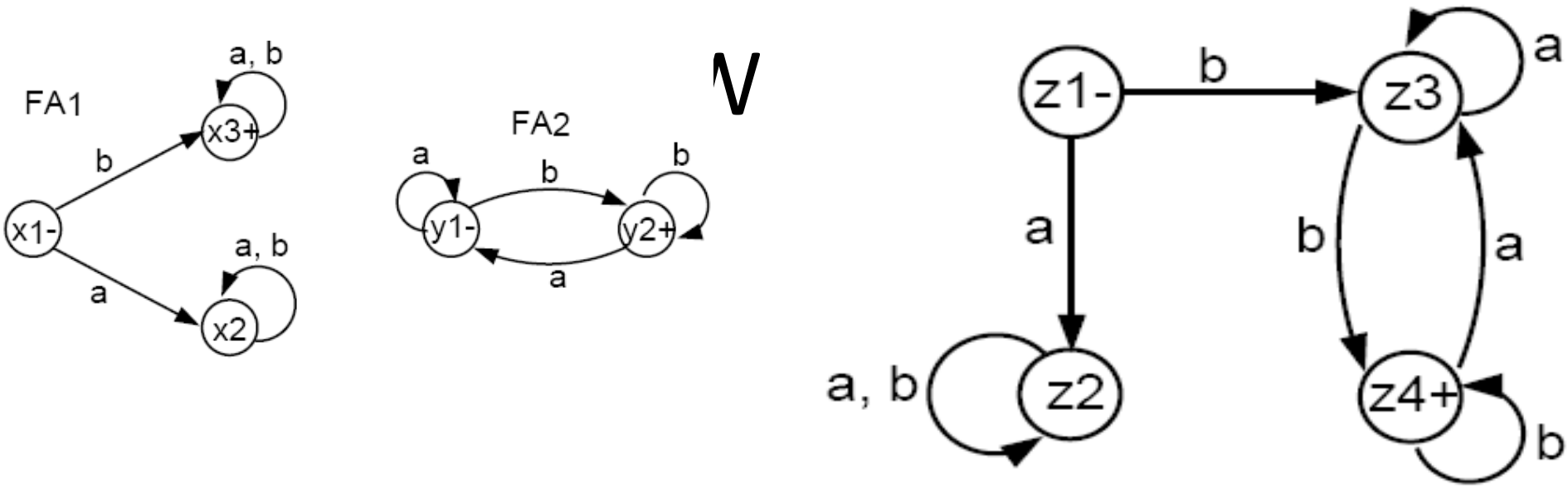


- In z3, if we read an a then the following happens. If z3 is x3, we can stay in x3 or jump to y1 (because x3 is a final state). If z3 is y1, we would loop back to y1. In any of the events, we end up at either x3 or y1, which is still z3. Hence, we have an a-loop at z3.

- In z3, if we read a b, then a different event takes place. If z3 is x3 we either stay in x3 or jump to y1. If z3 is y1, then we go to y2, a final state. Hence, we need a new final state z4 = x3 or y1 or y2.

- In z4, if we read an a, what happens? If z4 is x3 then we go back to x3 or jump to y1. If z4 is y1 then we loop back to y1. If z4 is y2, we go to y1. Thus, from z4, an a takes us to x3 or y1, which is z3.

- In z4, if we read a b, what happens? If z4 is x3, we go back to x3 or jump to y1. If z4 is y1, we go to y2, a final state. If z4 is y2, we loop back to y2, a final state. Hence, from z4 a b takes us to x3 or y1 or y2, which is still z4 (i.e., we have a b-loop here).



- This machine accepts all words that both begin and end with the letter b, which is what the product of the two languages (defined by FA_1 and FA_2 respectively) would be.

- If you multiply the two languages in opposite order (i.e. first FA_2 then FA_1), then the product language will be different. What is that language? Can you build a machine for that product language

NFA - Non-Deterministic Finite Automata

NFA

Definition: An NFA is a TG with a unique start state and with the property that each of its edge labels is a single alphabet letter.

- The regular deterministic finite automata are referred to as DFAs, to distinguish them from NFAs.
- As a TG, an NFA can have arbitrarily many a-edges and arbitrarily many b-edges coming out of each state.
- An input string is accepted by an NFA if there exists any possible path from - to +.

NFA

- An NFA is quintuple $M=\{Q,\Sigma,q_0, F, \delta\}$
  - Q is set of finite states
  - Σ is a finite set of symbols called ***alphabet***
  - q_0 belongs to Q is distinguished ***Start State***
  - F is subset of Q called the ***Final*** or Accepting states
  - δ is a total function from Q x Σ to P(Q) known as ***transition function***, such that, an input symbol may cause more than one next states, i.e., to one state out of a set of possible next states.
  - P(Q) is the power set of Q, that is, 2^Q

NFA

Definition: A nondeterministic finite automaton (or NFA) is a TG with a unique start state and with the property that each of its edge labels is a single alphabet letter.

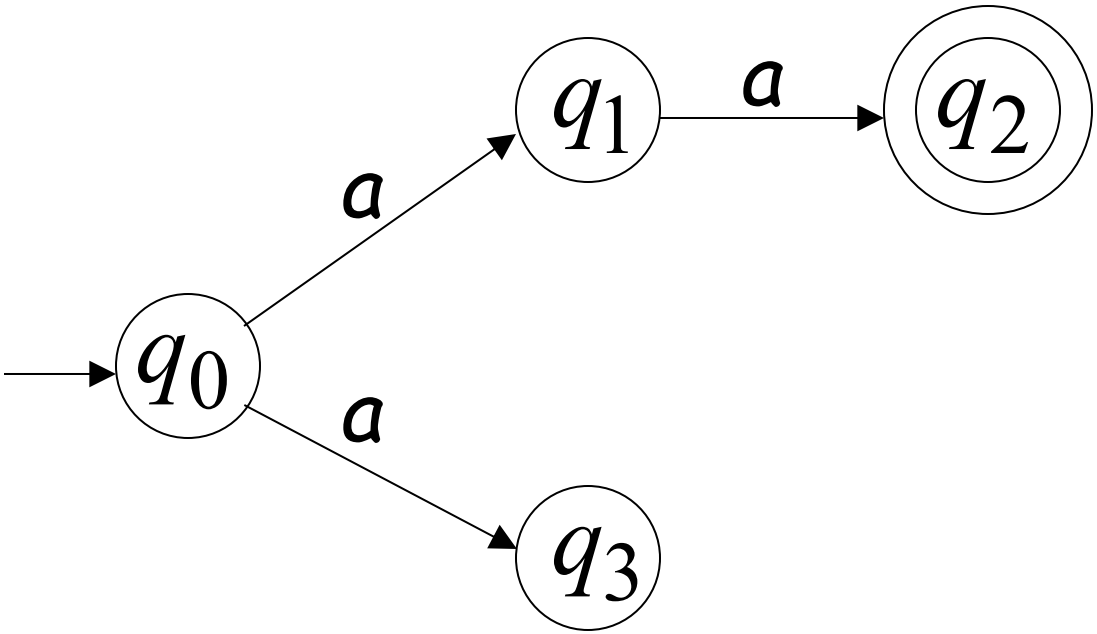
- The regular deterministic finite automata are referred to as DFAs, to distinguish them from NFAs.

- As a TG, an NFA can have arbitrarily many a-edges and arbitrarily many b-edges coming out of each state.

- An input string is accepted by an NFA if there exists any possible path from - to +.

Nondeterministic Finite Accepter (NFA)

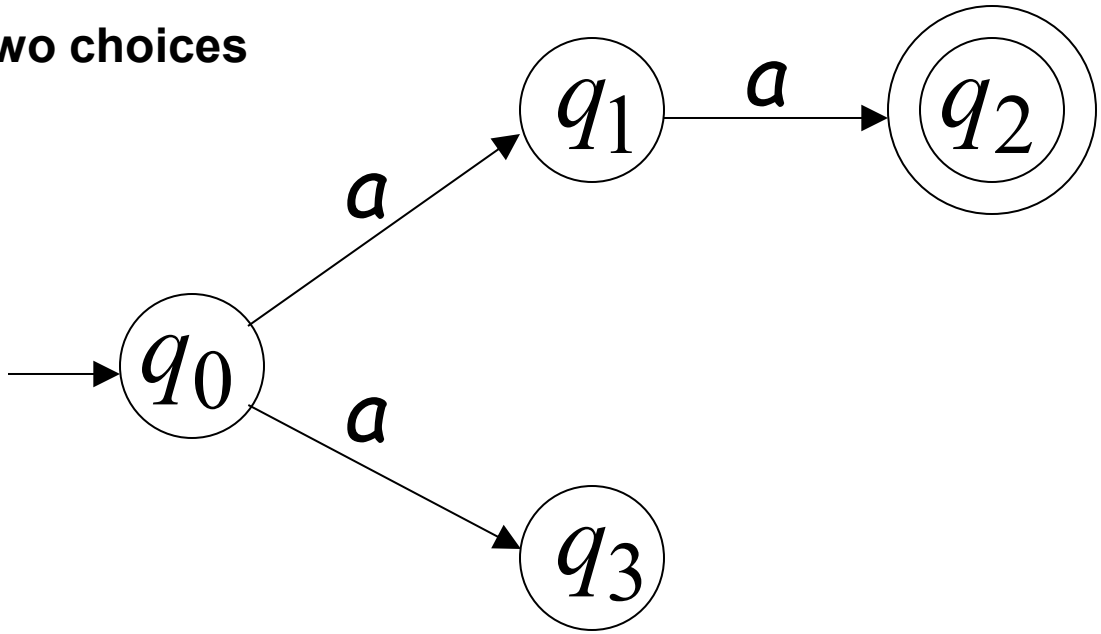
Alphabet = $\{a\}$



Nondeterministic Finite Accepter (NFA)

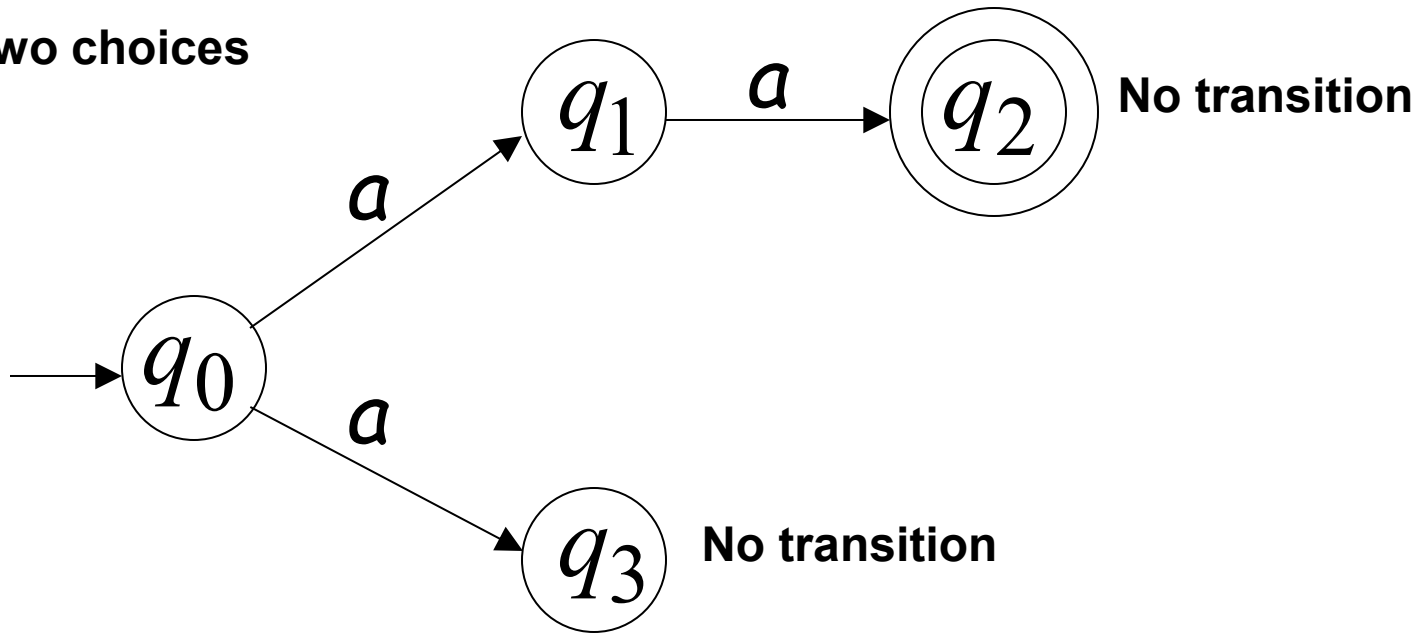
Alphabet = $\{a\}$

Two choices

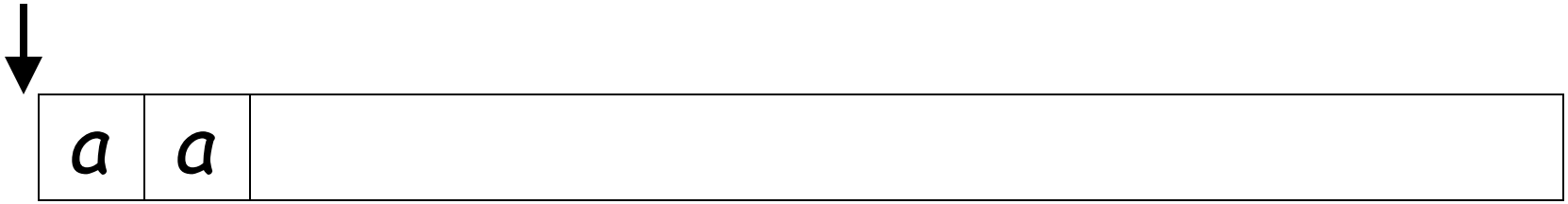


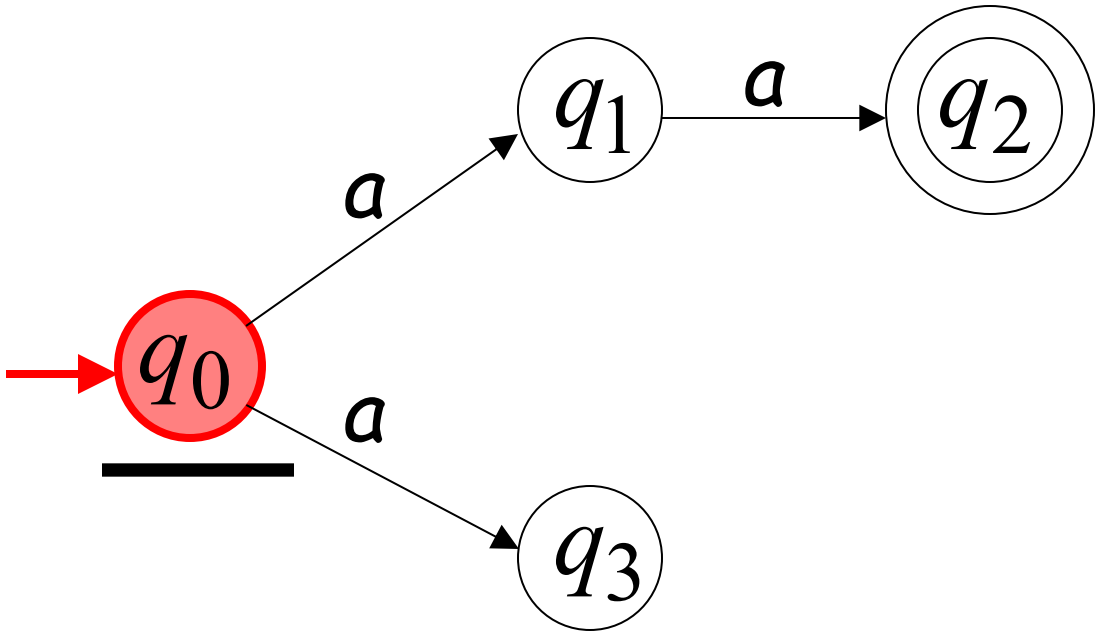
Nondeterministic Finite Accepter (NFA)

Alphabet = $\{a\}$

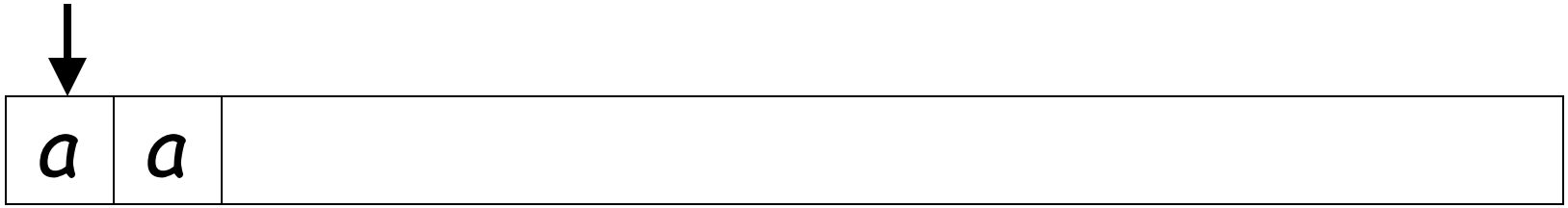


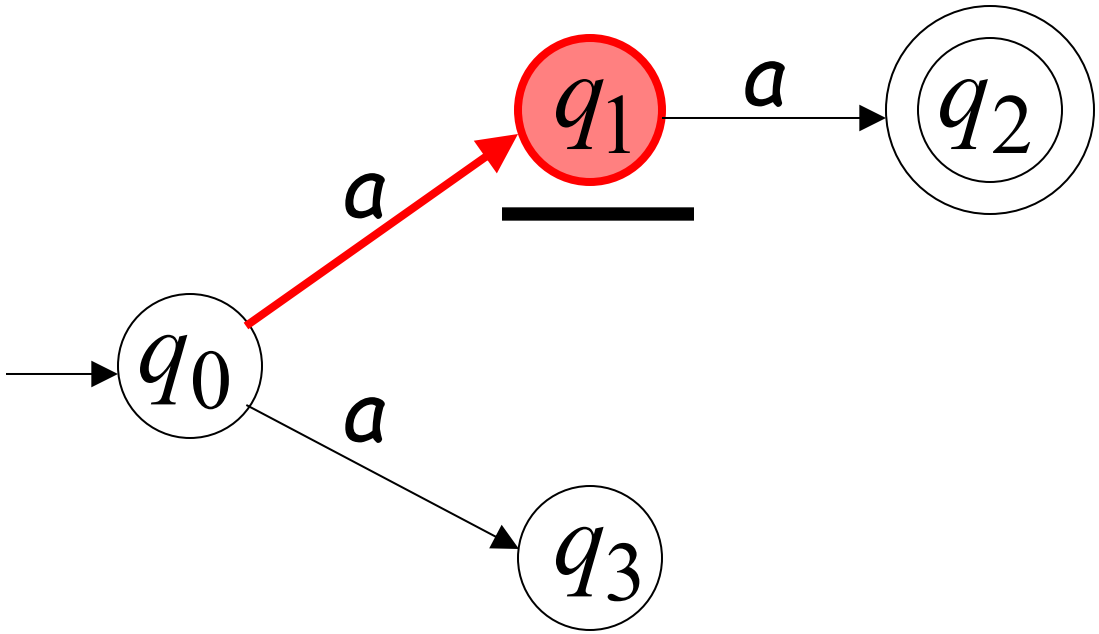
First Choice



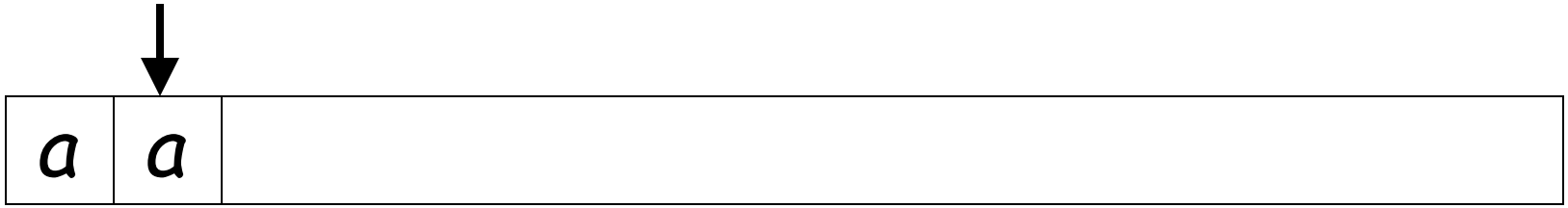


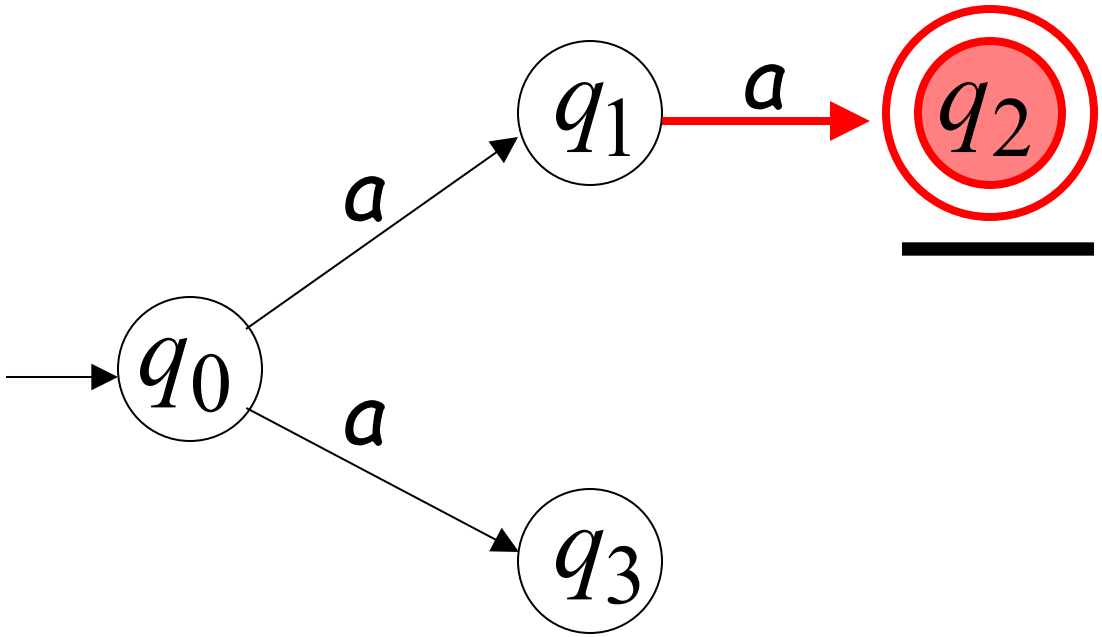
First Choice



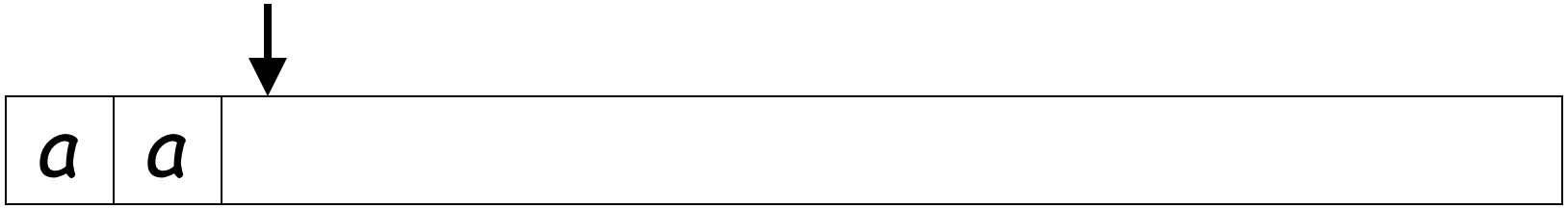


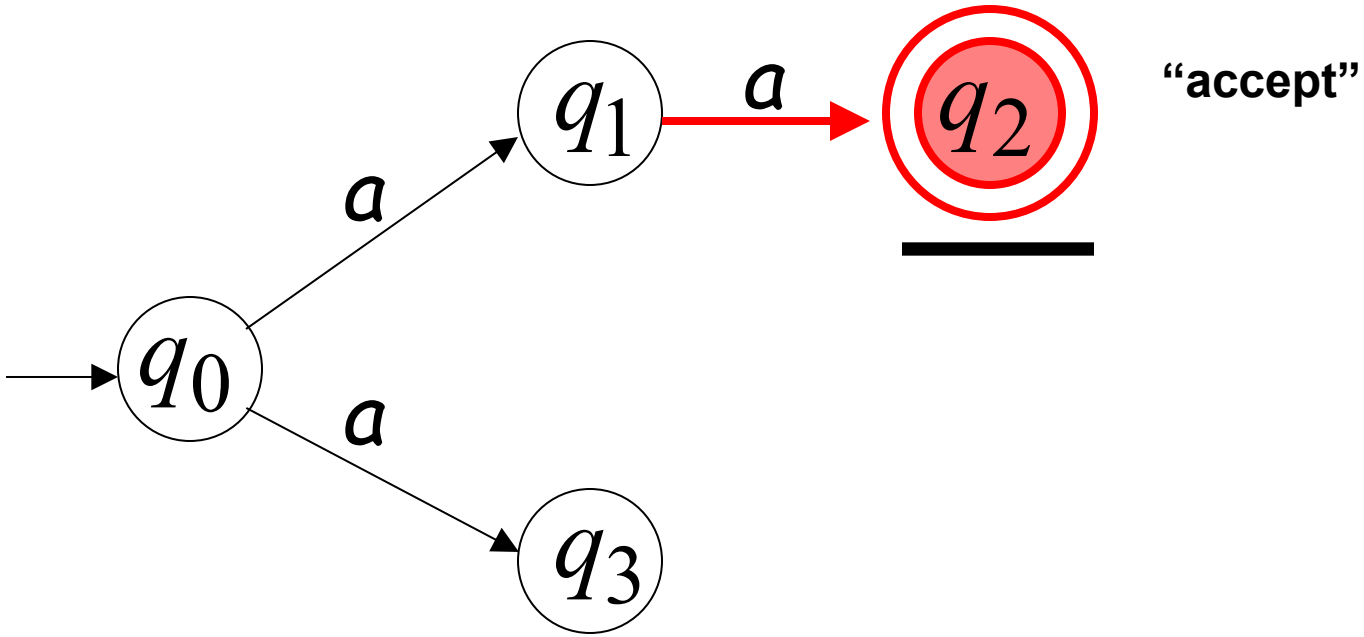
First Choice



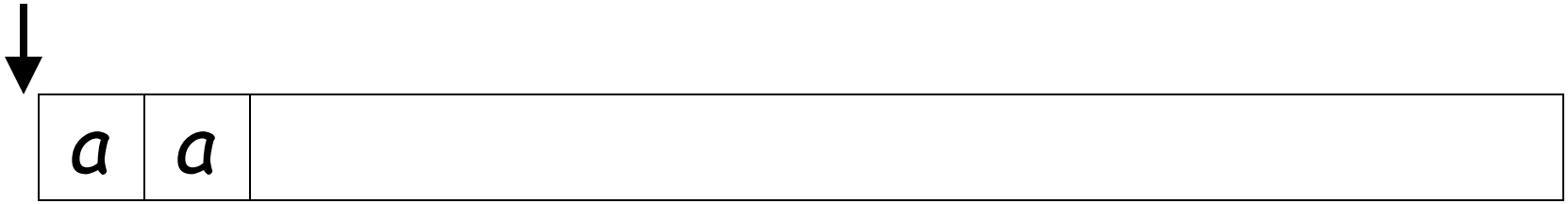


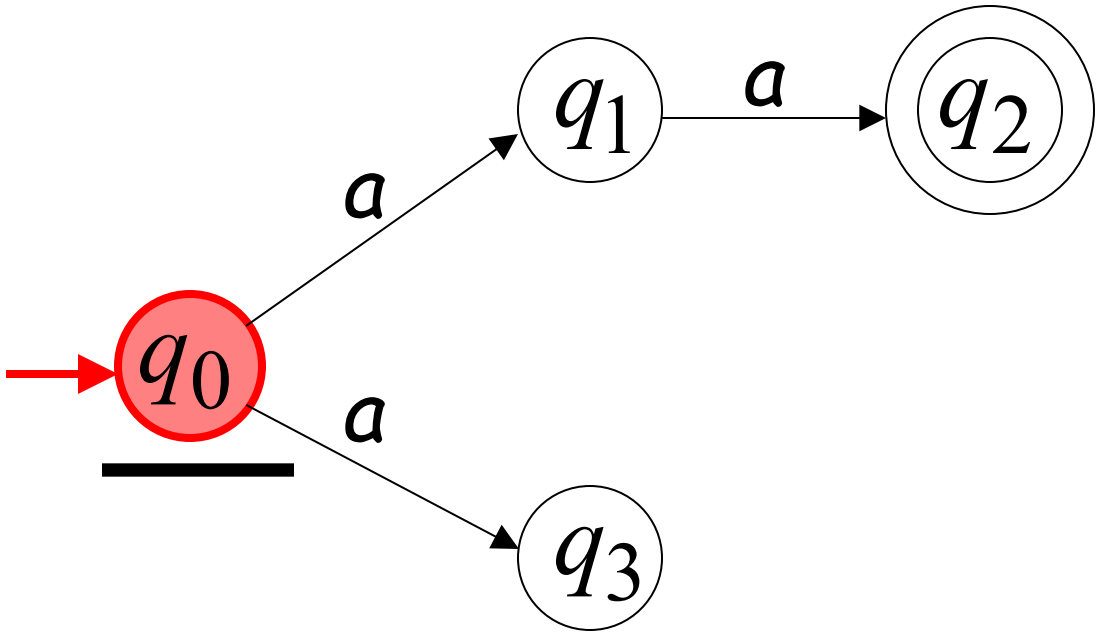
First Choice



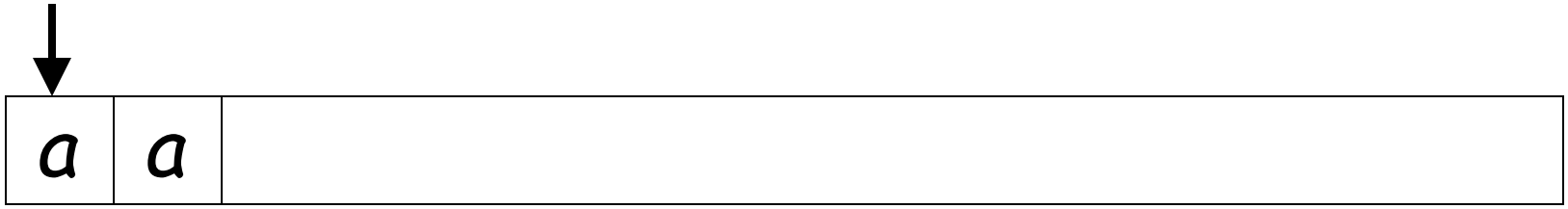


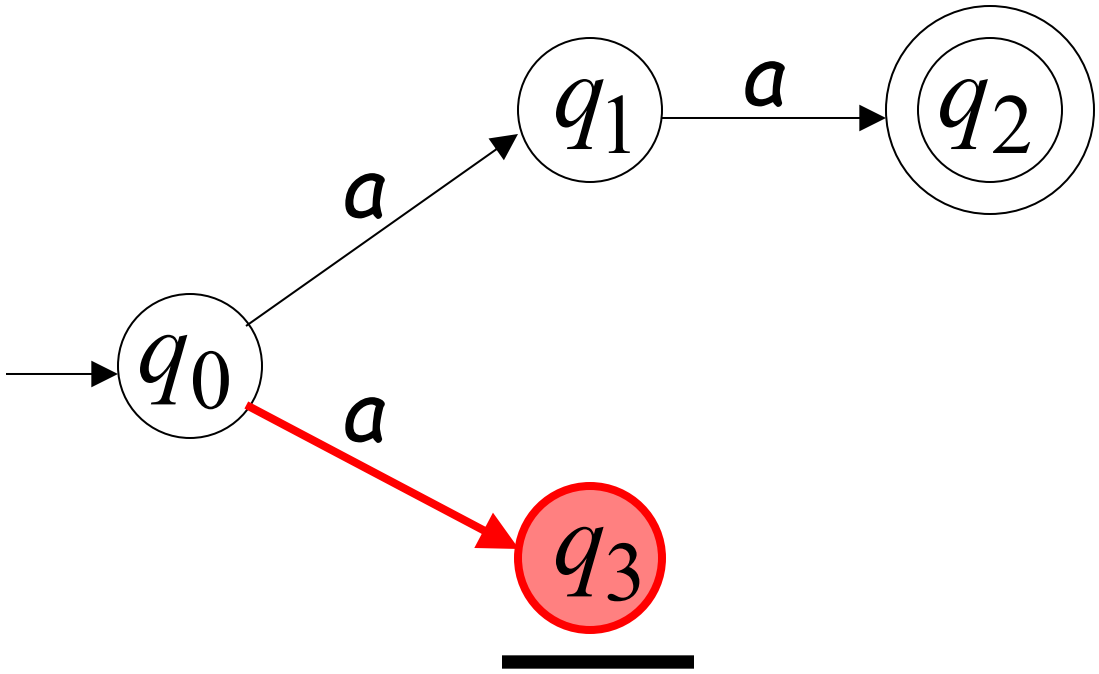
Second Choice



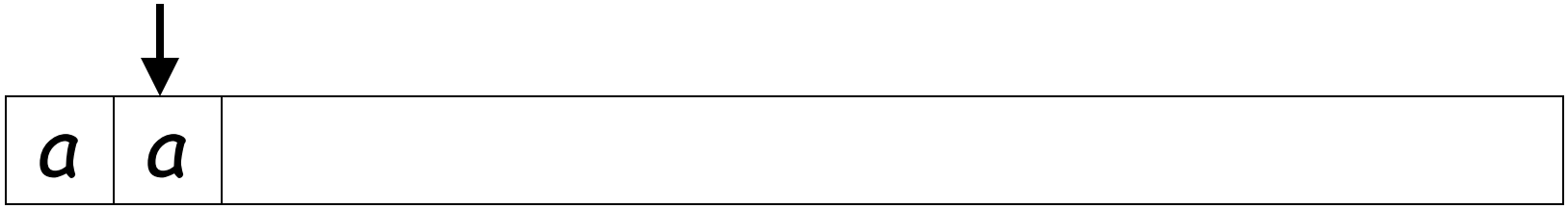


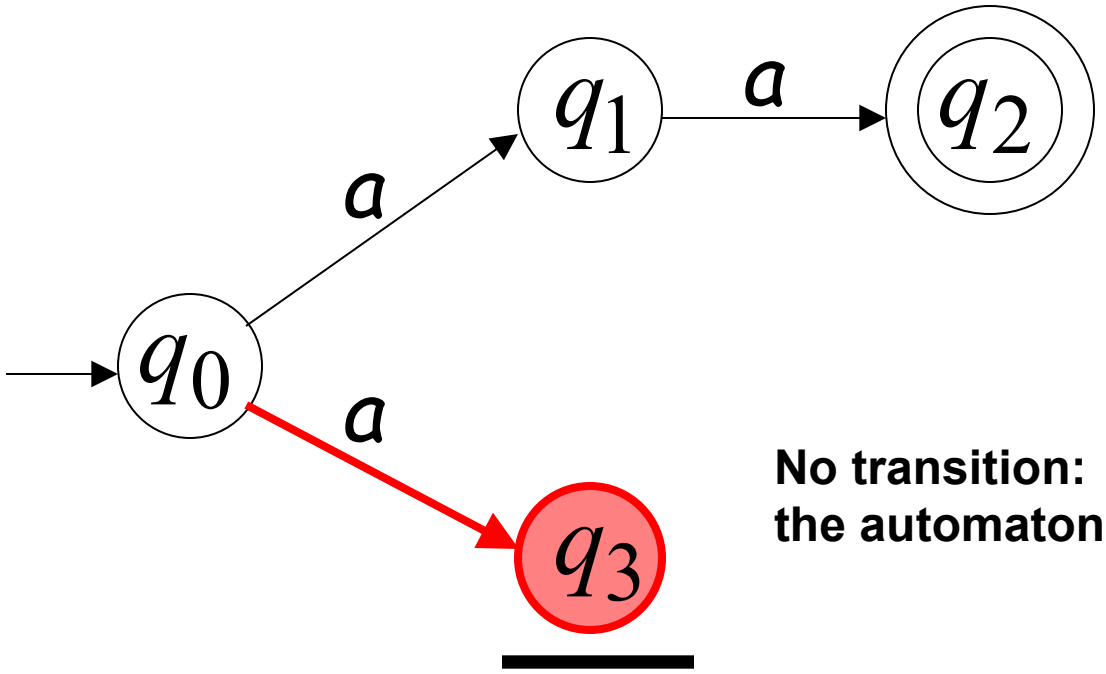
Second Choice





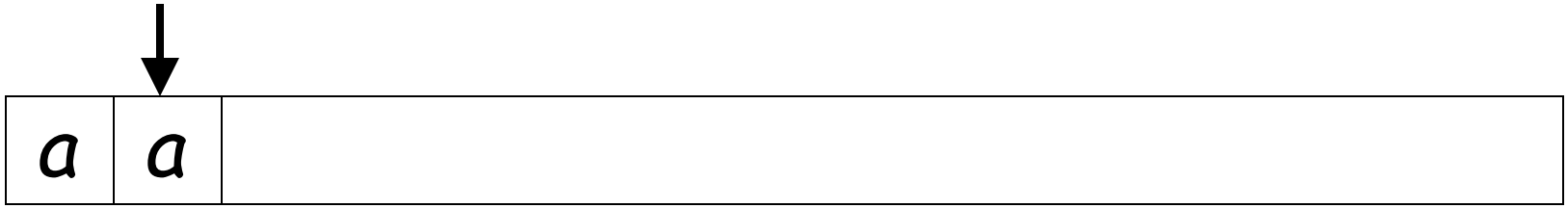
Second Choice

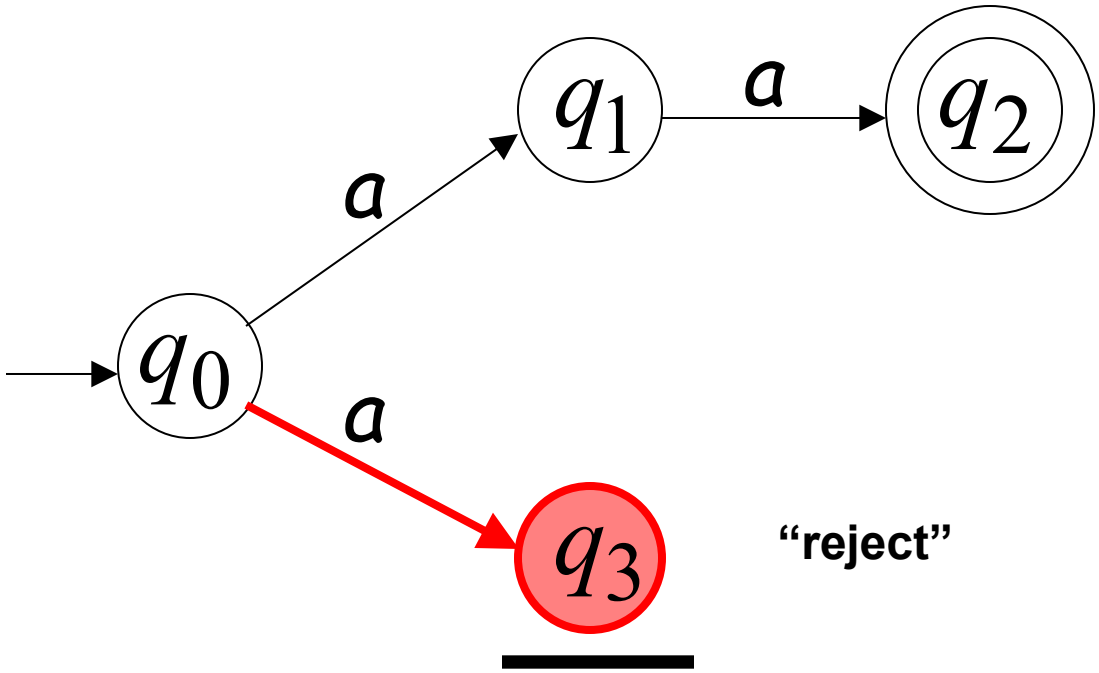




No transition:
the automaton hangs

Second Choice





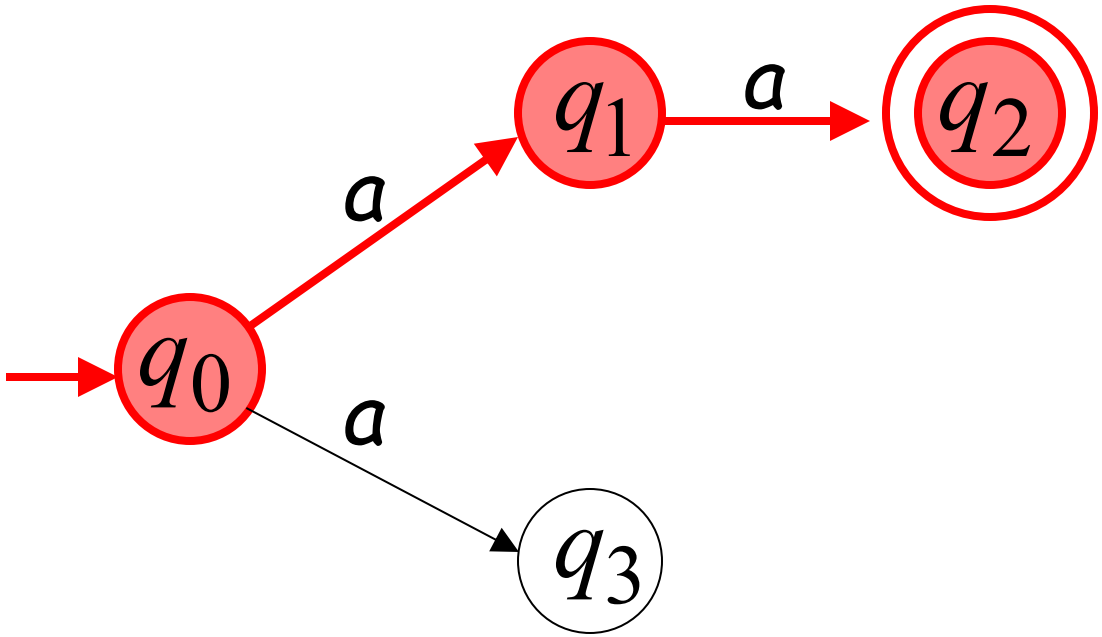
Observation

An NFA accepts a string:

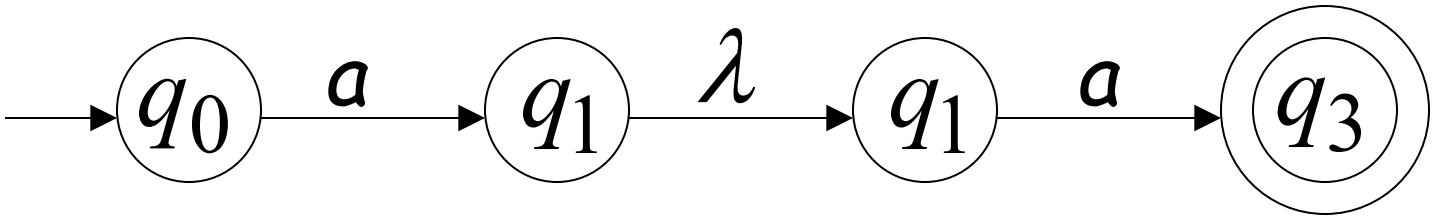
if
there is a computation of the NFA
that accepts the string

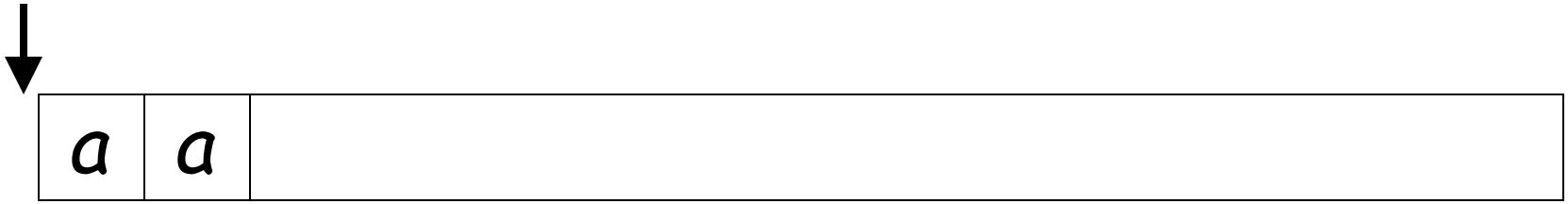
Example

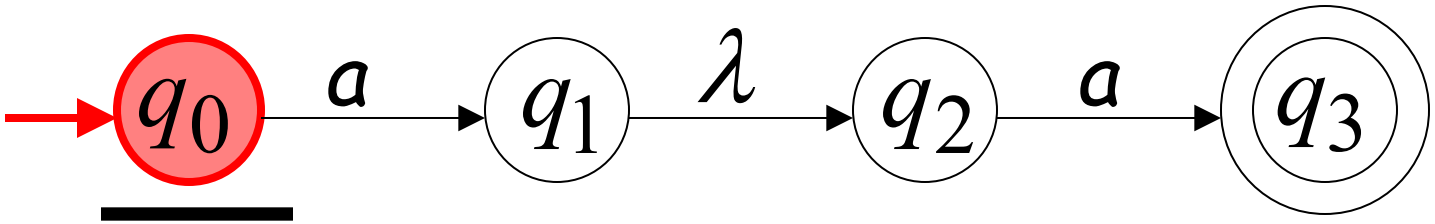
aa **is accepted by the NFA:**

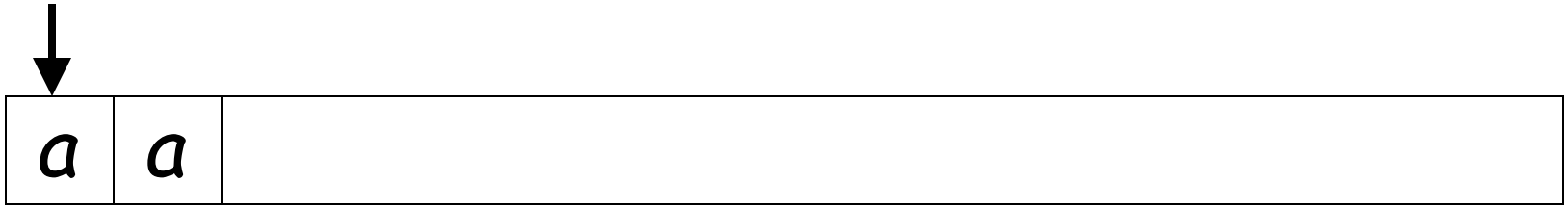


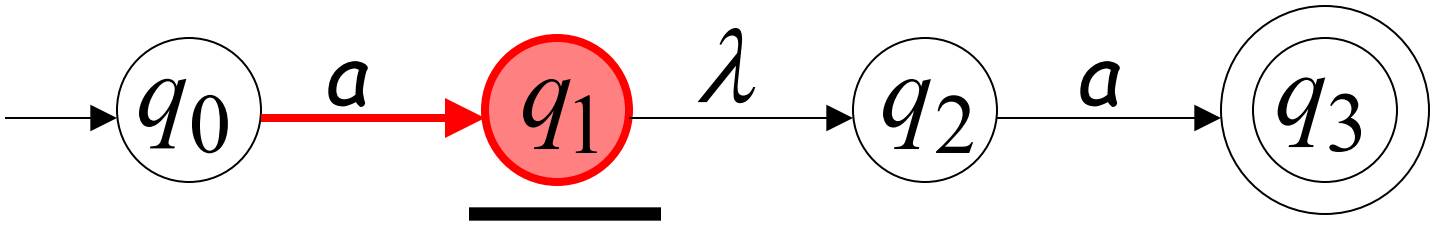
Lambda Transitions



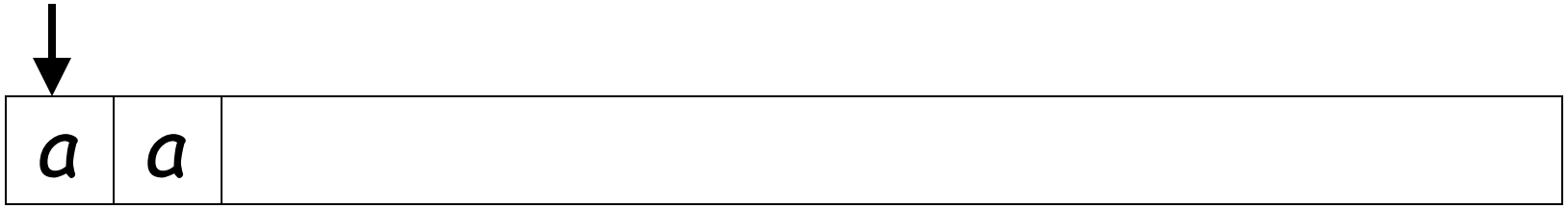

a
a

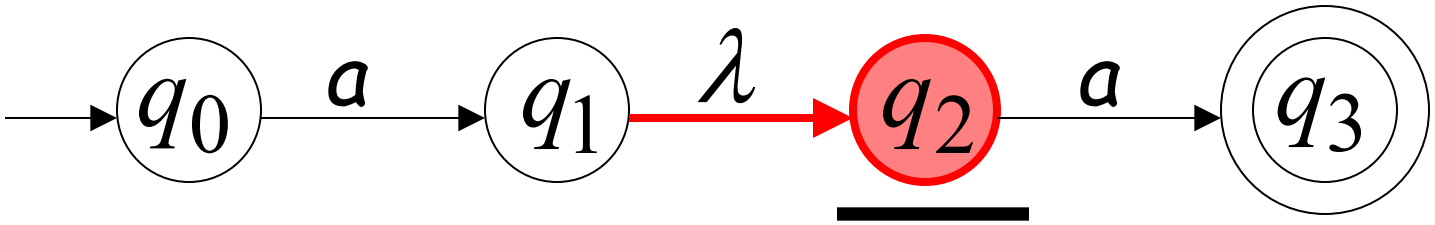

q_0
a
q_1
λ
q_2
a
q_3

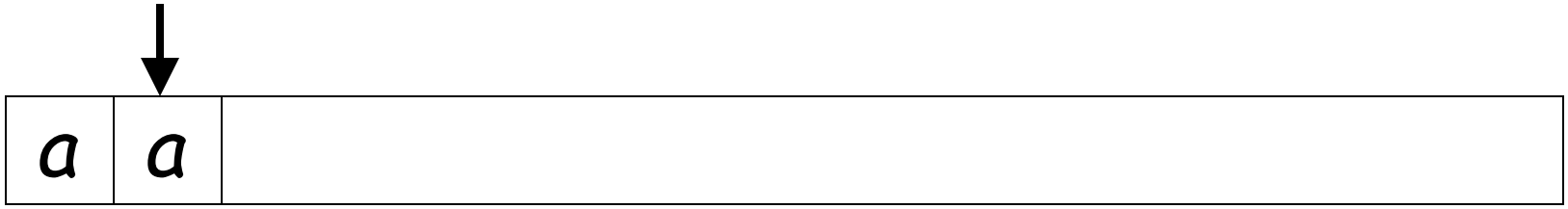

a
a

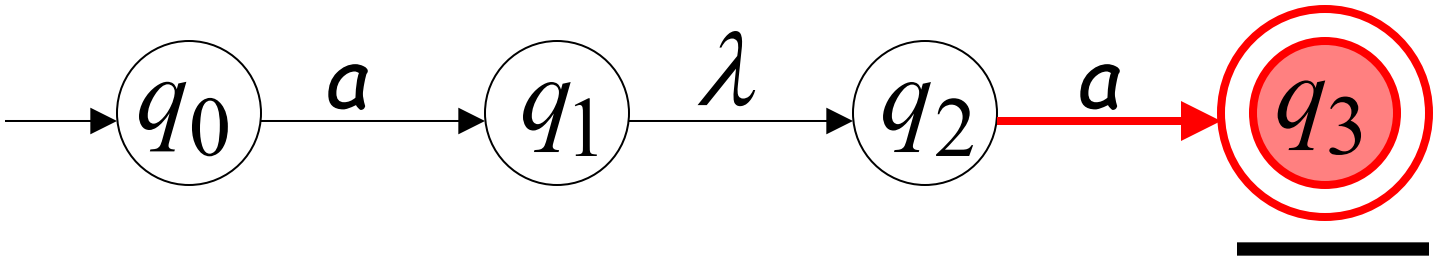

q_0
a
q_1
λ
q_2
a
q_3

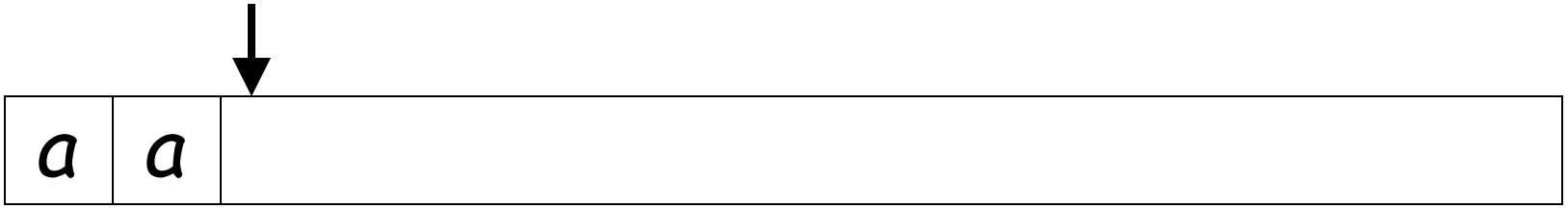
(read head doesn't move)

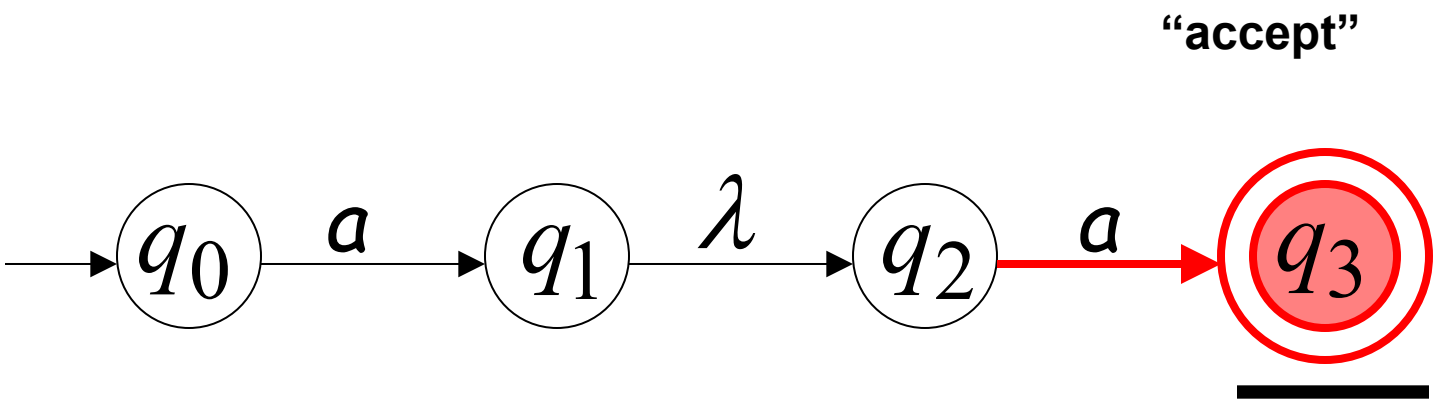





a
a


q_0
a
q_1
λ
q_2
a
q_3

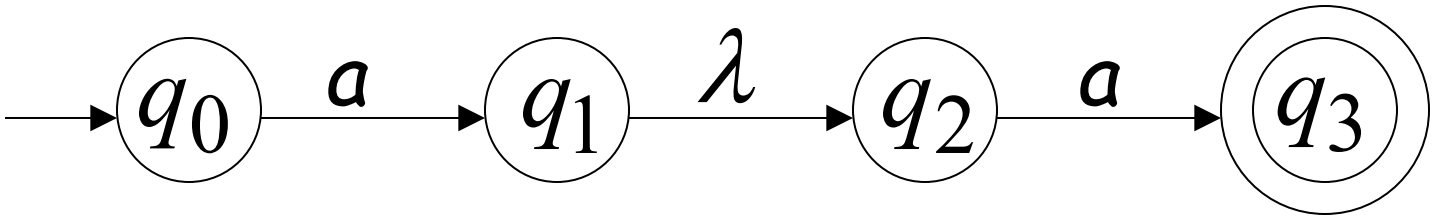




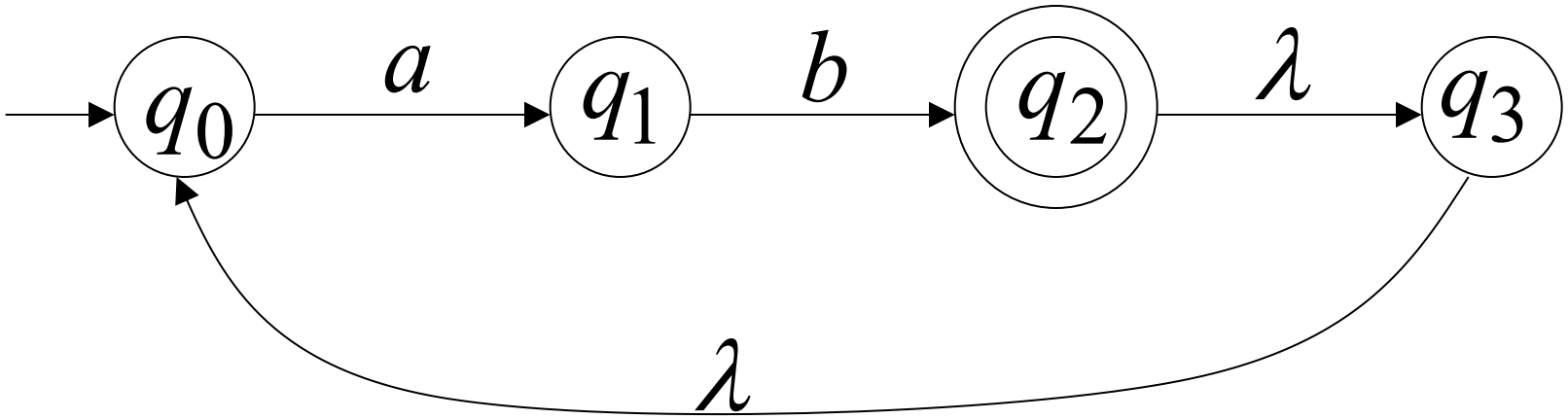
String aa is accepted

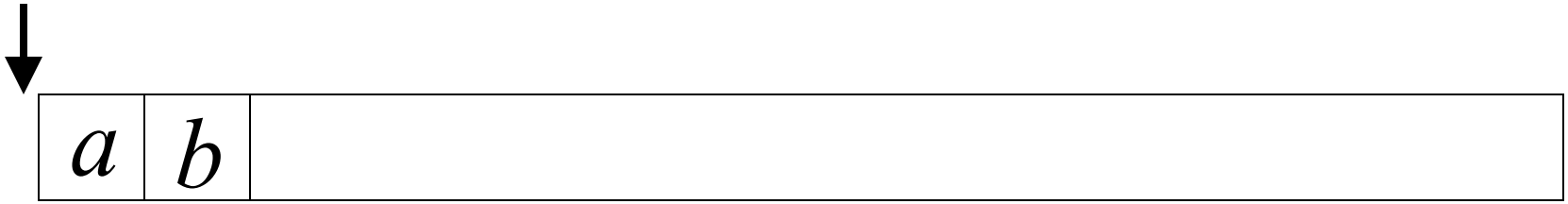
Language accepted:

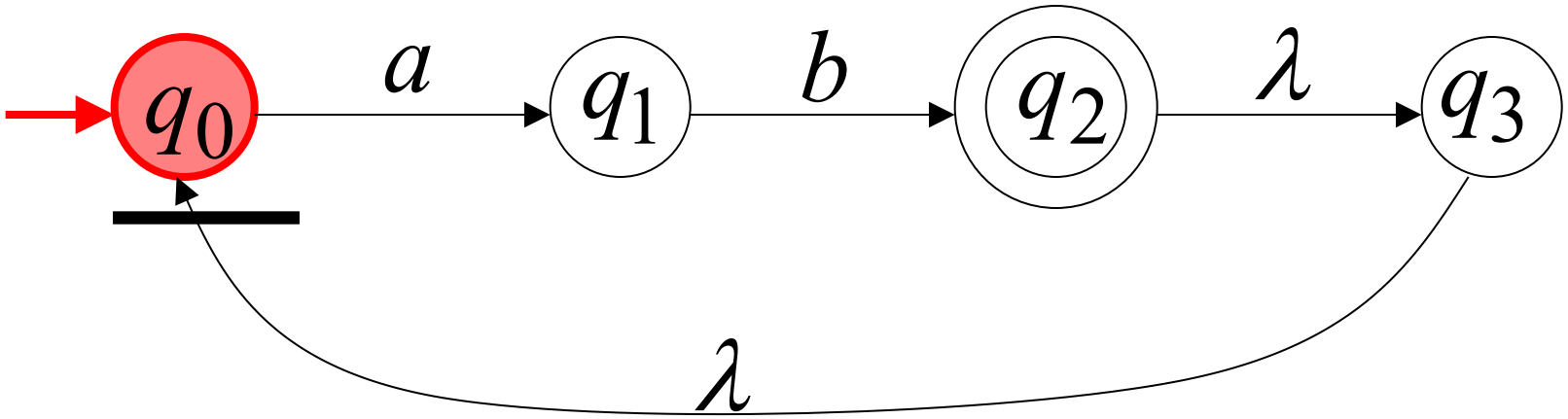
$$L = \{aa\}$$

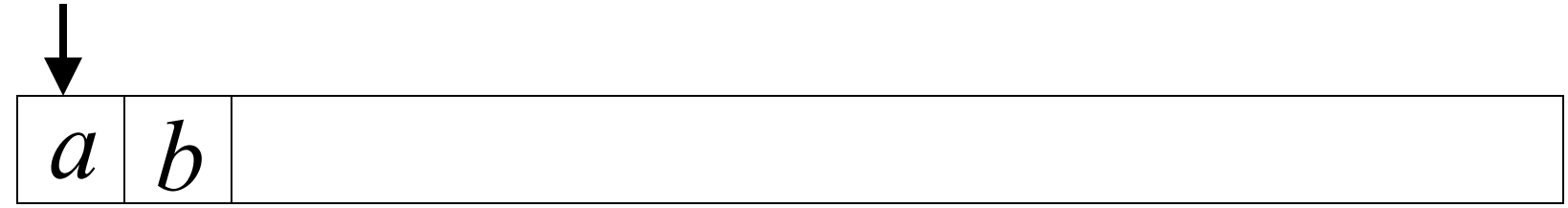


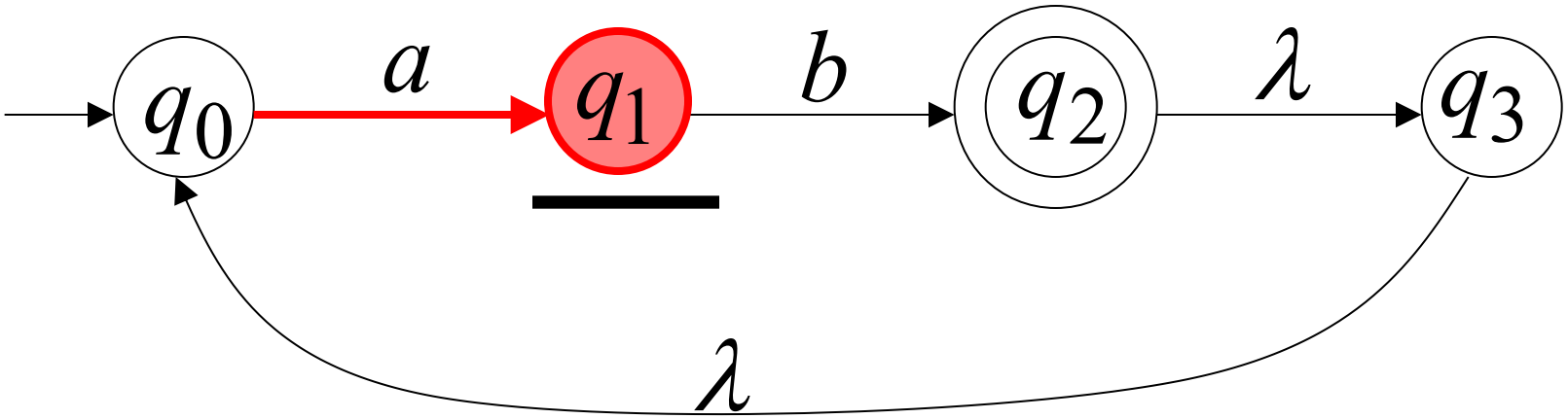
Another NFA Example

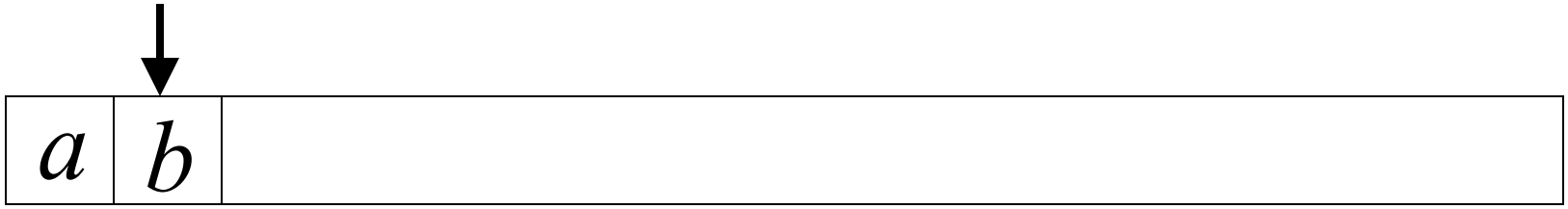


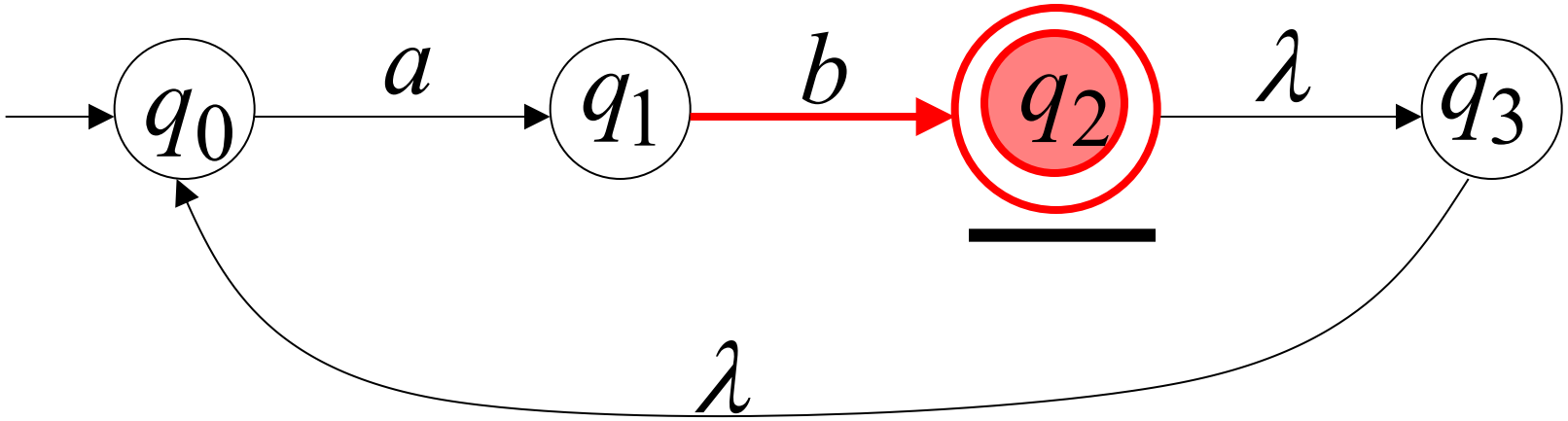

a
b

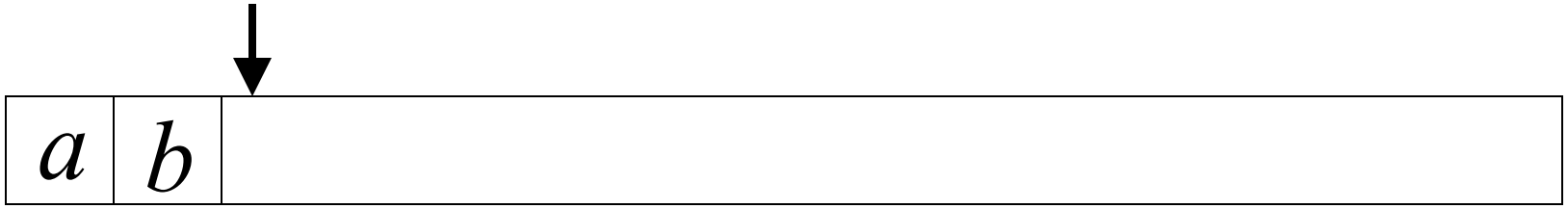

q_0
a
q_1
b
q_2
λ
q_3
λ

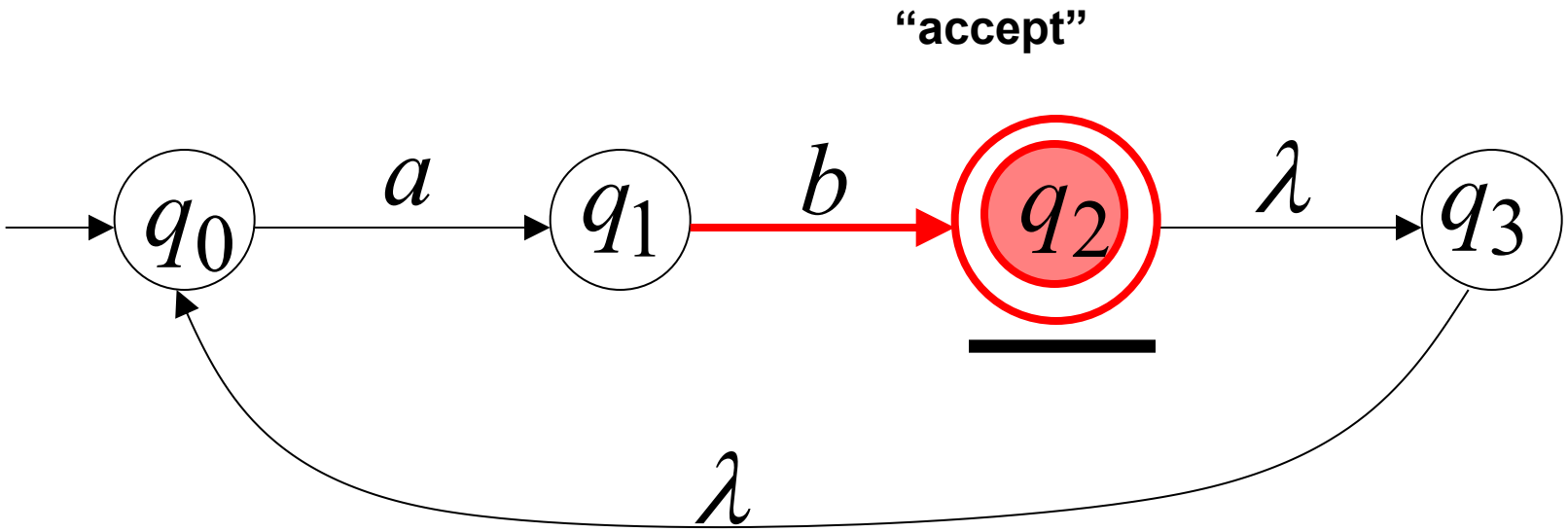

a
b


q_0
a
q_1
b
q_2
λ
q_3
λ

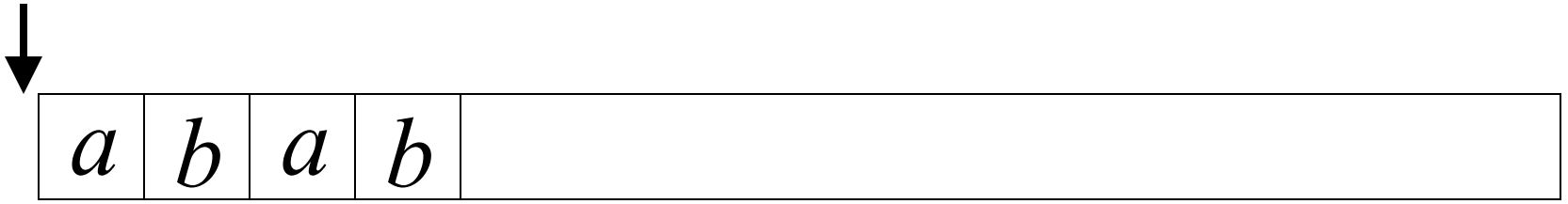

a
b

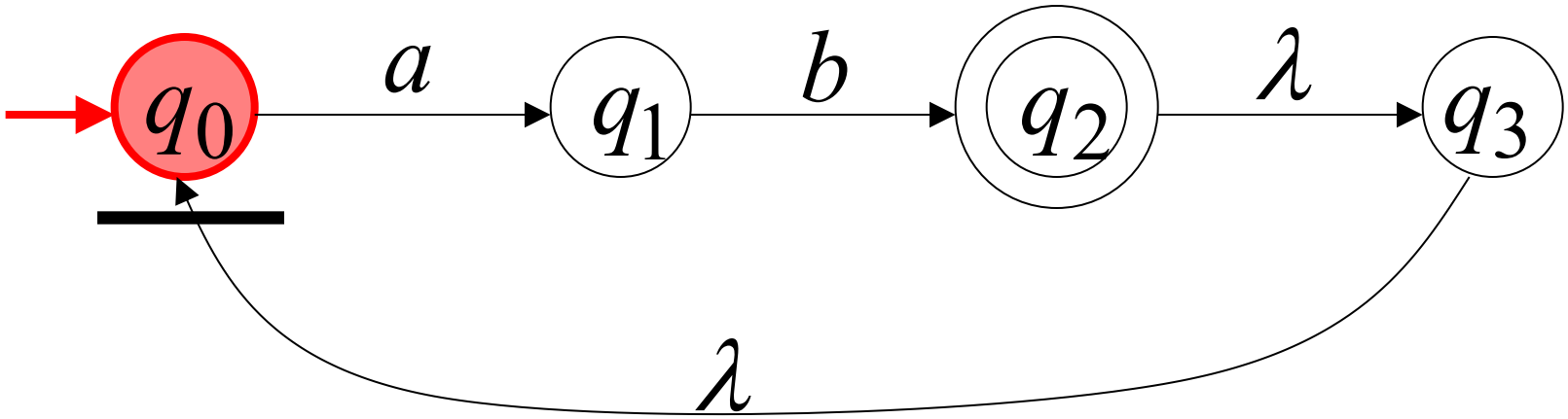

q_0
a
q_1
b
q_2
λ
q_3
λ

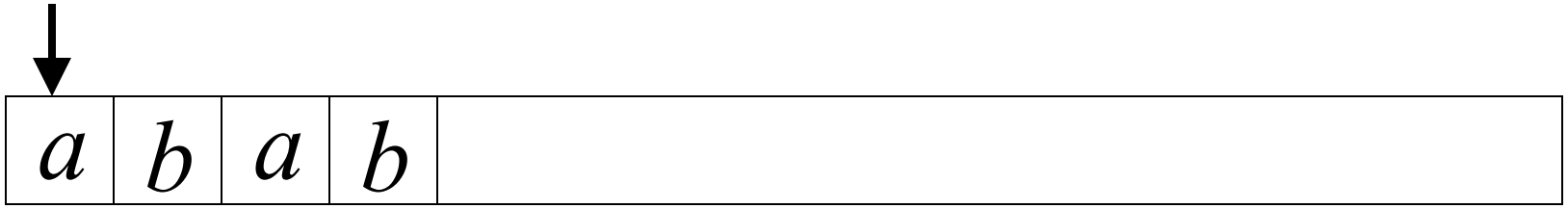

a
b

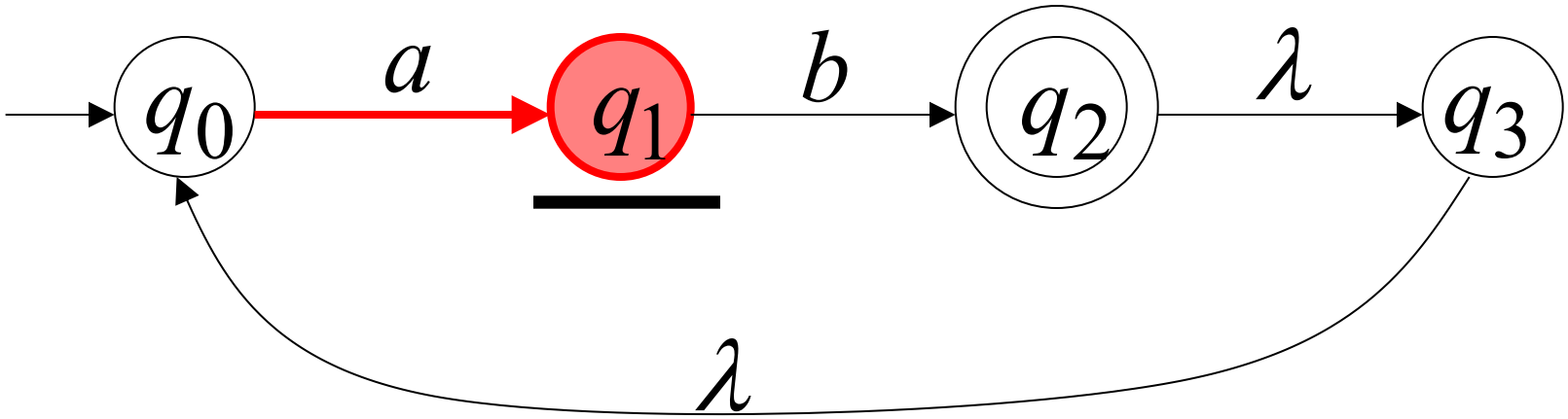

"accept"
q_0
q_1
q_2
q_3
a
b
λ
λ

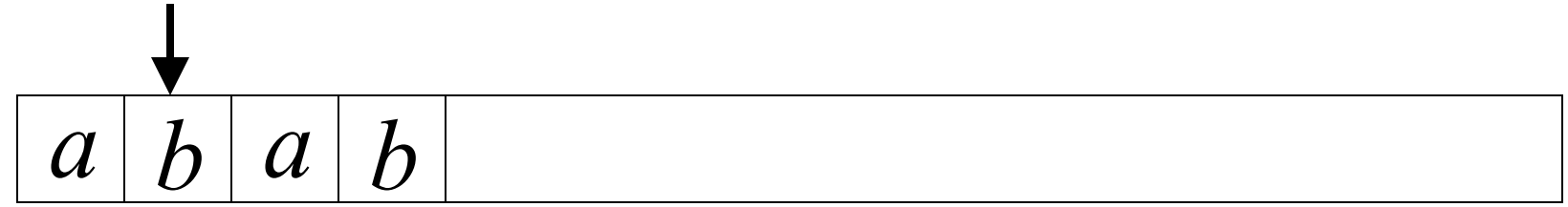
Another String

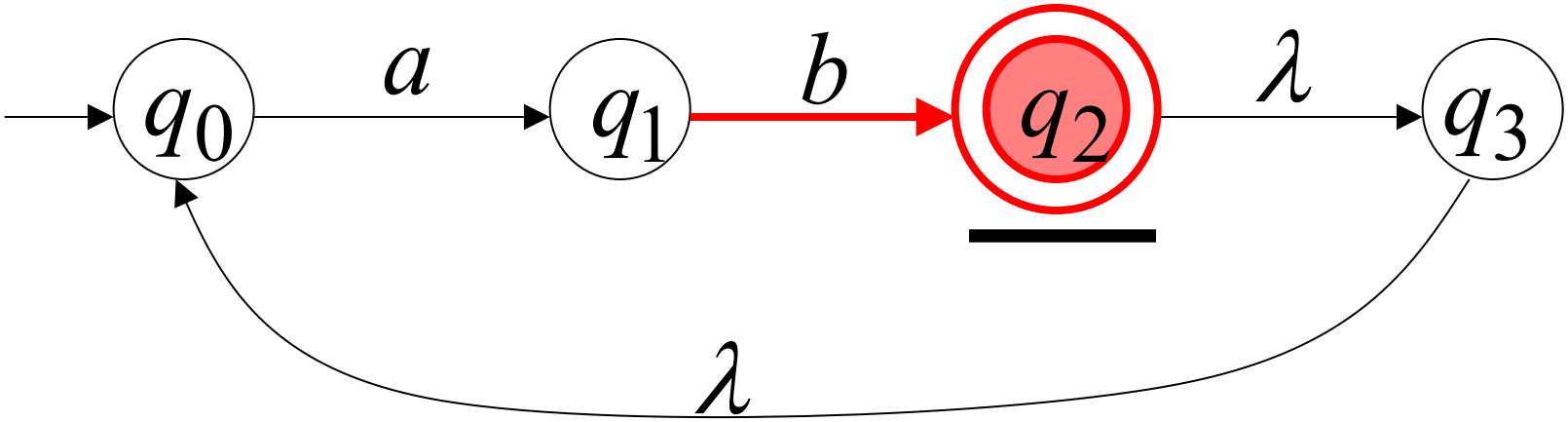


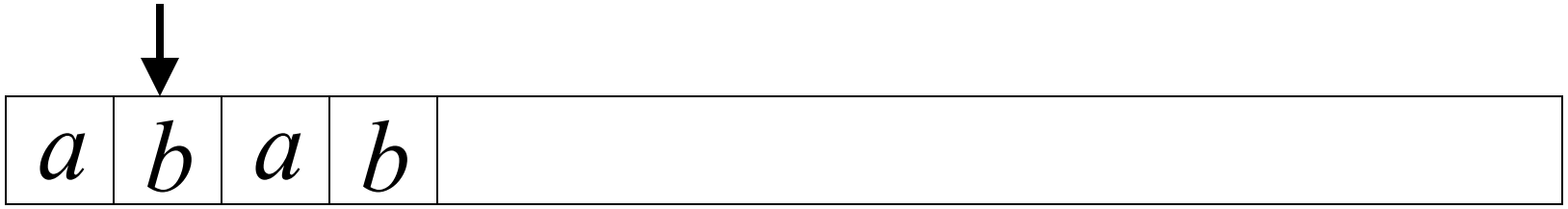


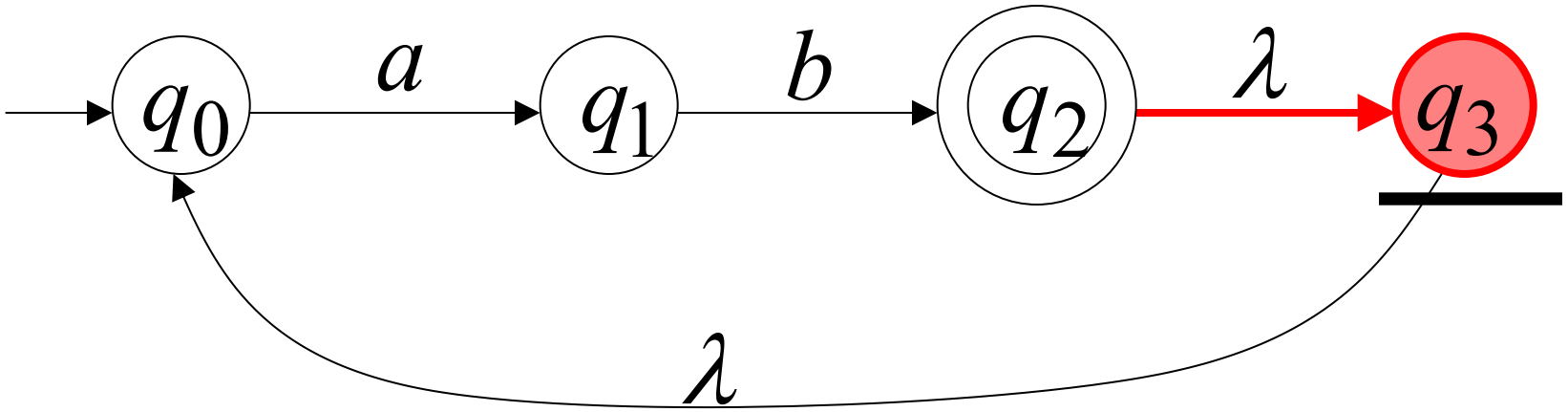

a
b
a
b

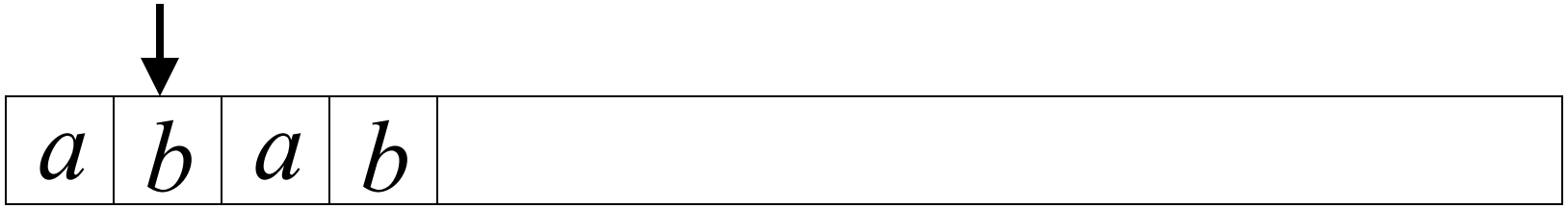

q_0
a
q_1
b
q_2
λ
q_3
λ

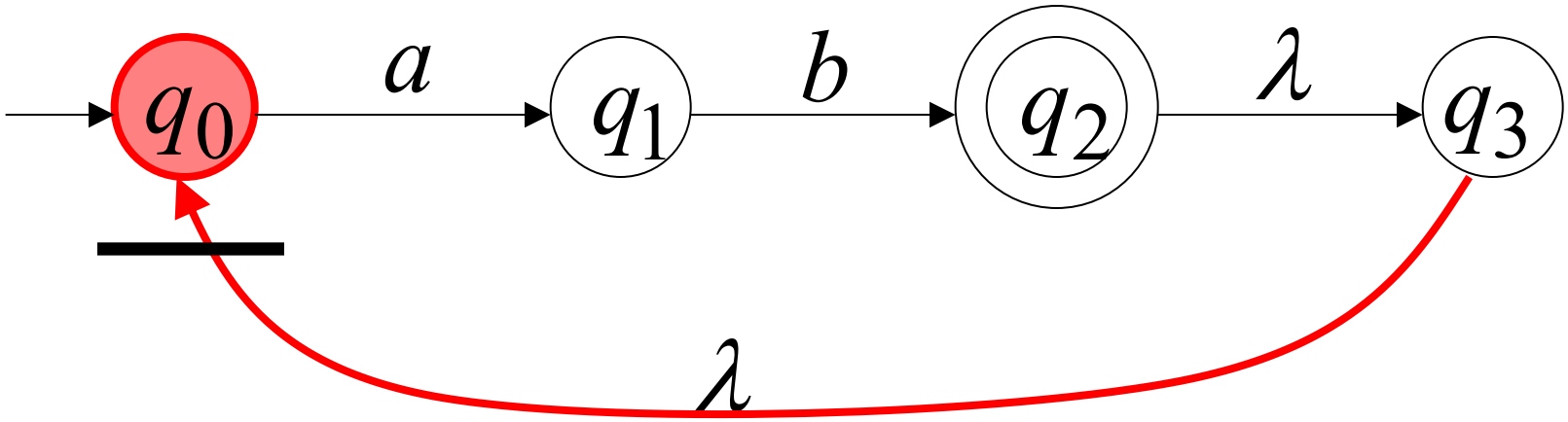

a
b
a
b

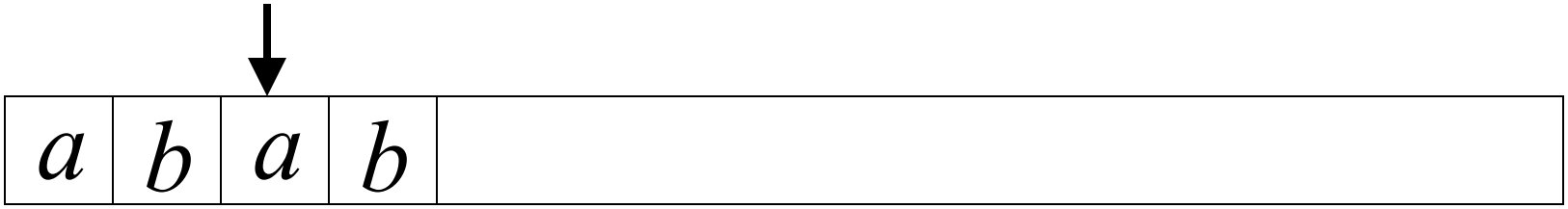

q_0
a
q_1
b
q_2
λ
q_3
λ

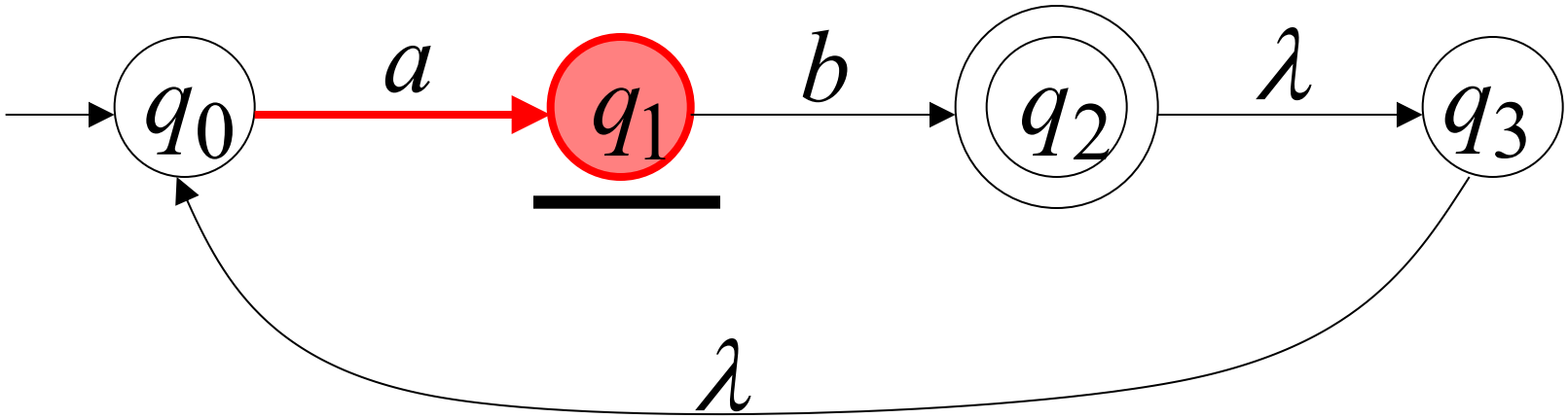

a
b
a
b

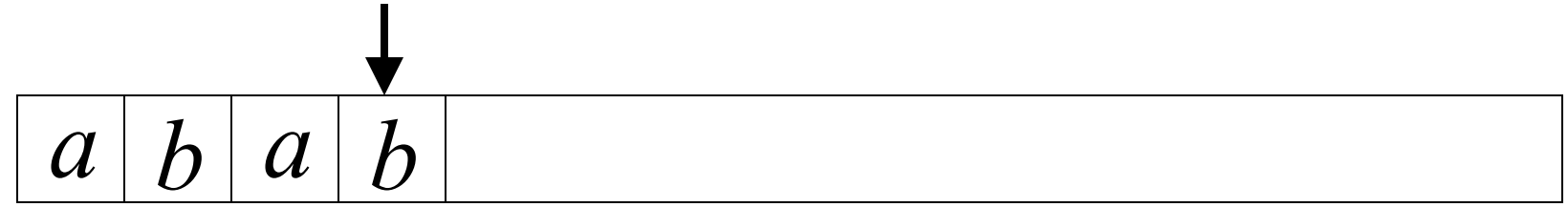

q_0
a
q_1
b
q_2
λ
q_3
λ

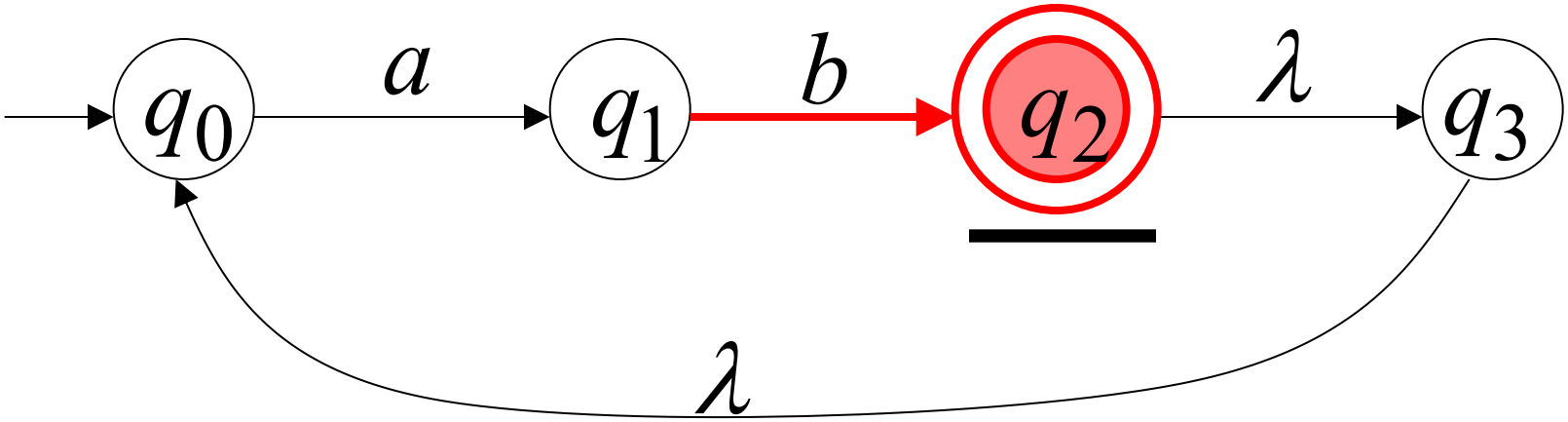

a
b
a
b

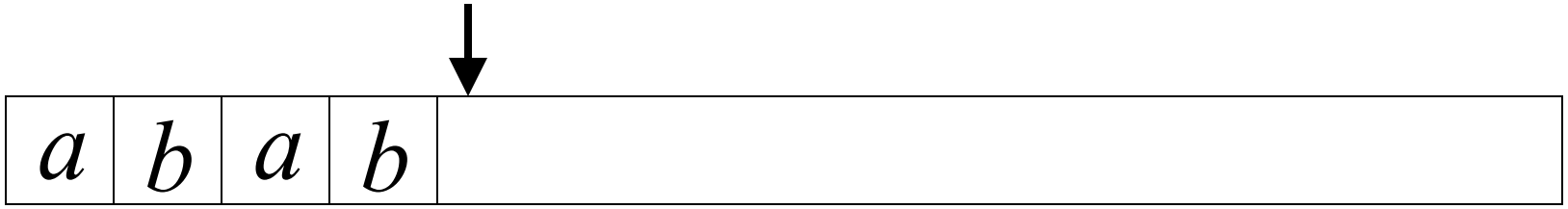

q_0
a
q_1
b
q_2
λ
q_3
λ

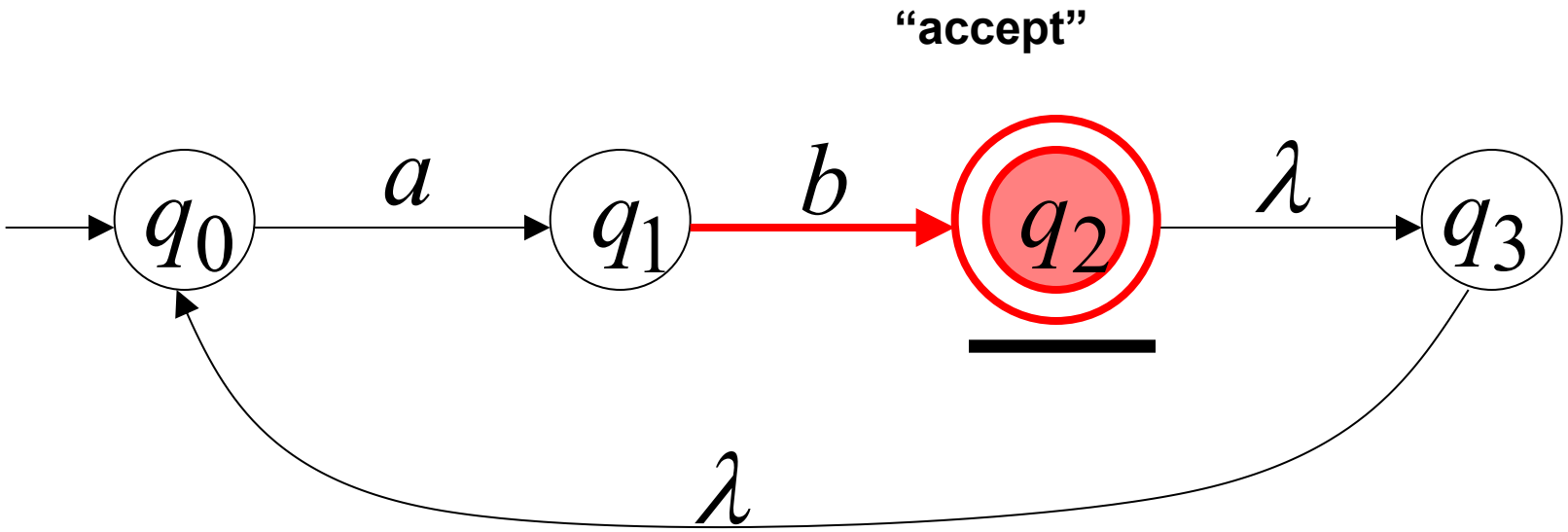

a
b
a
b


q_0
a
q_1
b
q_2
λ
q_3
λ


a
b
a
b


q_0
a
q_1
b
q_2
λ
q_3
λ

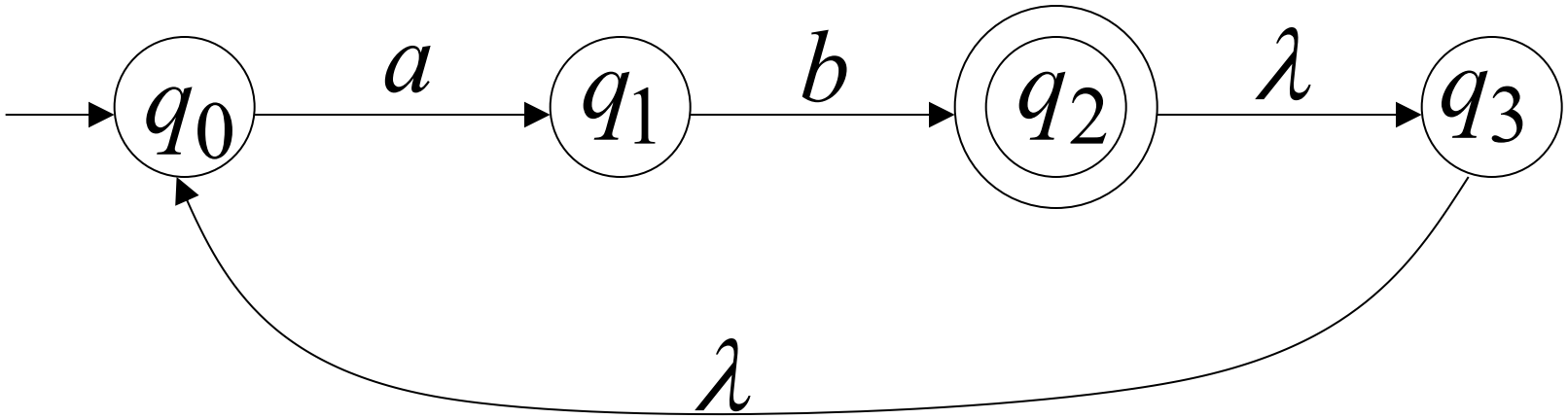

a
b
a
b


"accept"
q_0
q_1
q_2
q_3
a
b
λ
λ

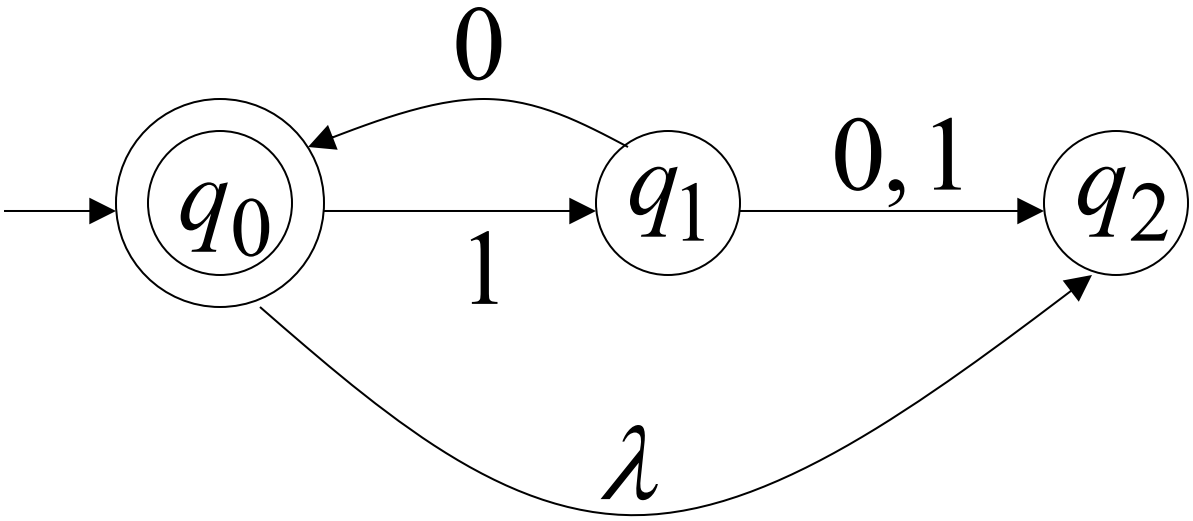
Language accepted

$$L = \{ab,\ abab,\ ababab,\ ...\}$$

$$= \{ab\}^{+}$$

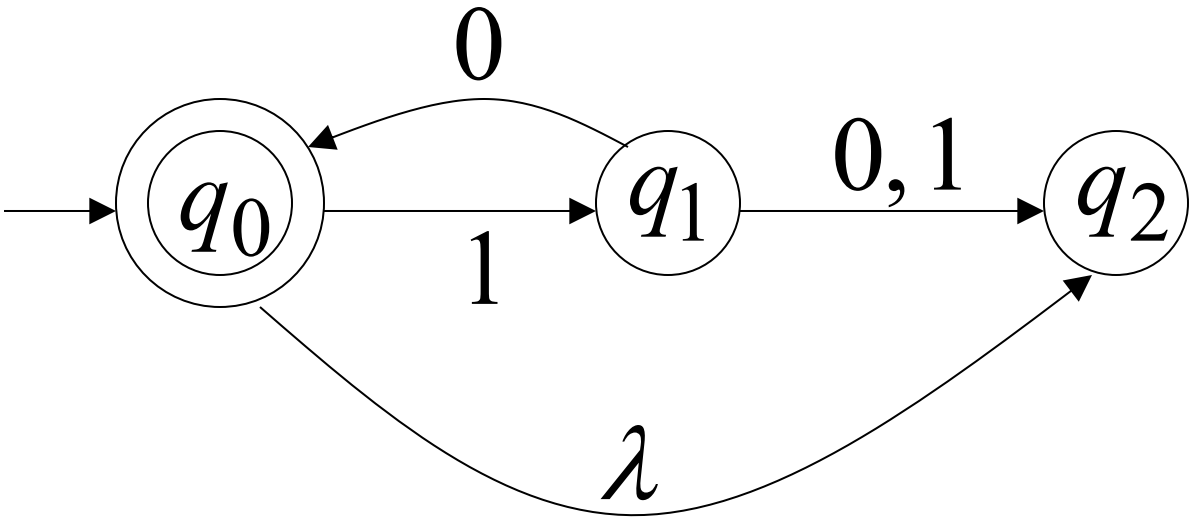


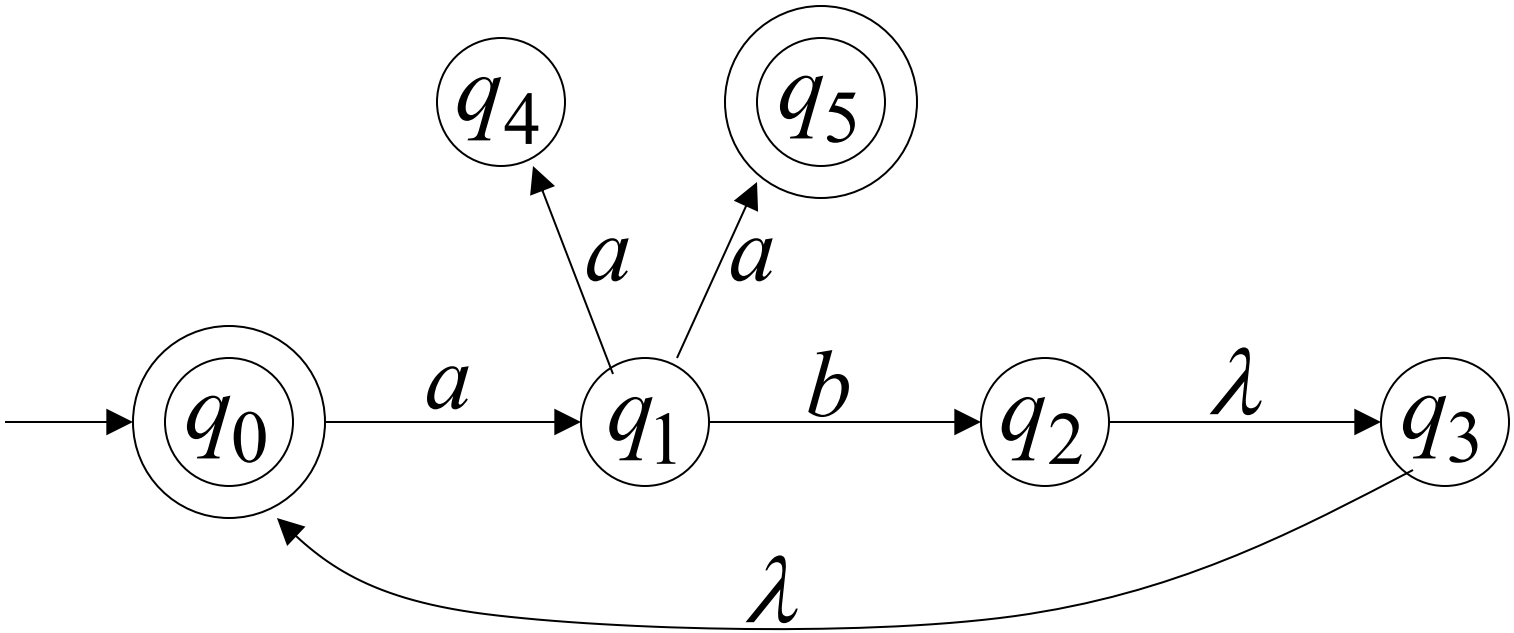
Another NFA Example



Language accepted

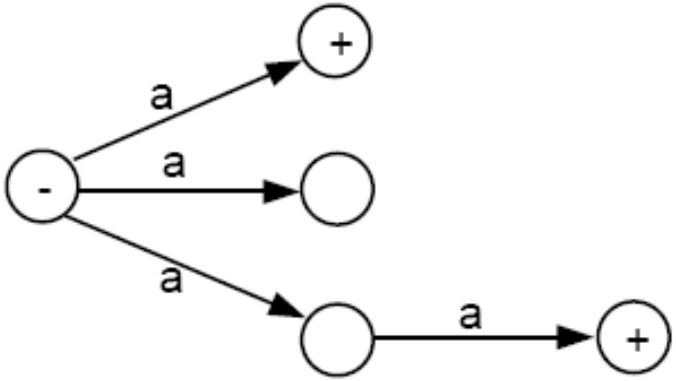
$$L = \{\lambda,\ 10,\ 1010,\ 101010,\ ...\}$$
$$= \{10\}^*$$

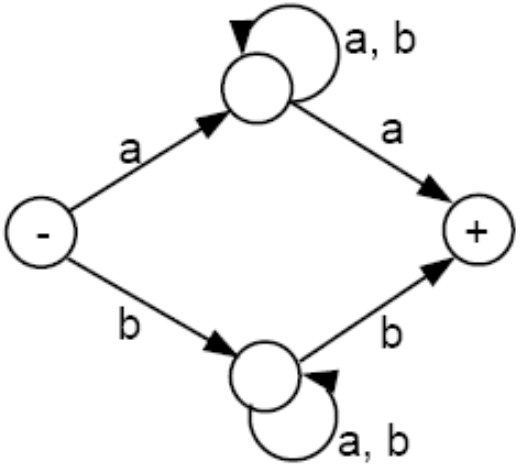


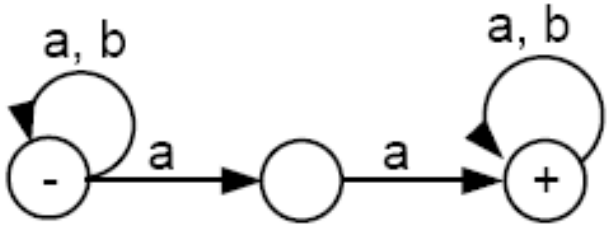


$$L(M) = \{aa\} \cup \{ab\}^* \cup \{ab\}^+\{aa\}$$

Examples of NFAs







Theorem 7

for every NFA, there is some FA that accepts exactly the same language.

- **Proof 1**

- By the proof of part 2 of Kleene's theorem, we can convert an NFA into a regular expression, since an NFA is a TG.

- By the proof of part 3 of Kleene's theorem, we can construct an FA that accepts the same language as the regular expression. Hence, for every

- NFA, there is a corresponding FA.

Distinguished Features

FA/DFA

1. One start state ONLY & Zero or more Final States

NFA

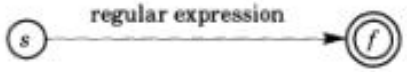
1. One start state ONLY & Zero or More Final States
2. Null transitions
3. Non-determinism

TG

1. One or More Start States
2. Zero or More Final States
3. Multiple letters on the edges
4. Null transitions
5. Non-determinism

Regular Expression to Finite Automaton (5.4)

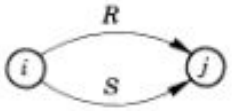
Given a regular expression, we start the algorithm with a machine that has a start state, a single final state, and an edge labeled with the given regular expression as follows:



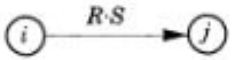
Now transform this machine into a DFA or an NFA by applying the following rules until all edges are labeled with either a letter or Λ:

1. If an edge is labeled with ∅, then erase the edge.
2. Transform any diagram like

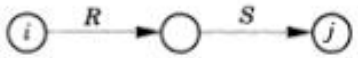
into the diagram



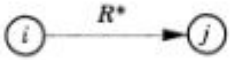
3. Transform any diagram like



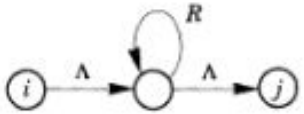
into the diagram



4. Transform any diagram like



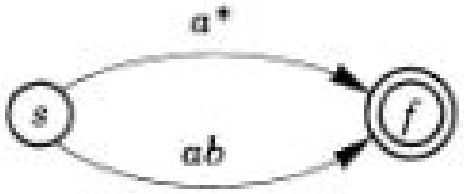
into the diagram



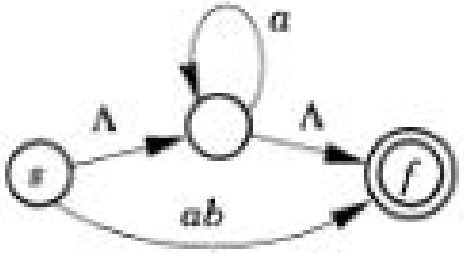
End of Algorithm

EXAMPLE 4. To construct an NFA for $a^* + ab$, we'll start with the diagram

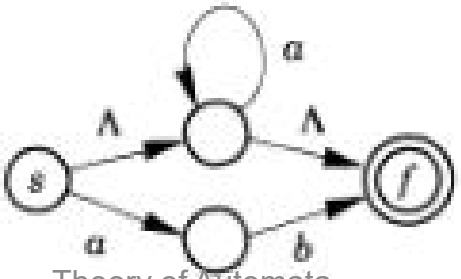
Next we apply rule 2 to obtain the following NFA:



Next we'll apply rule 4 to a^* to obtain the following NFA:



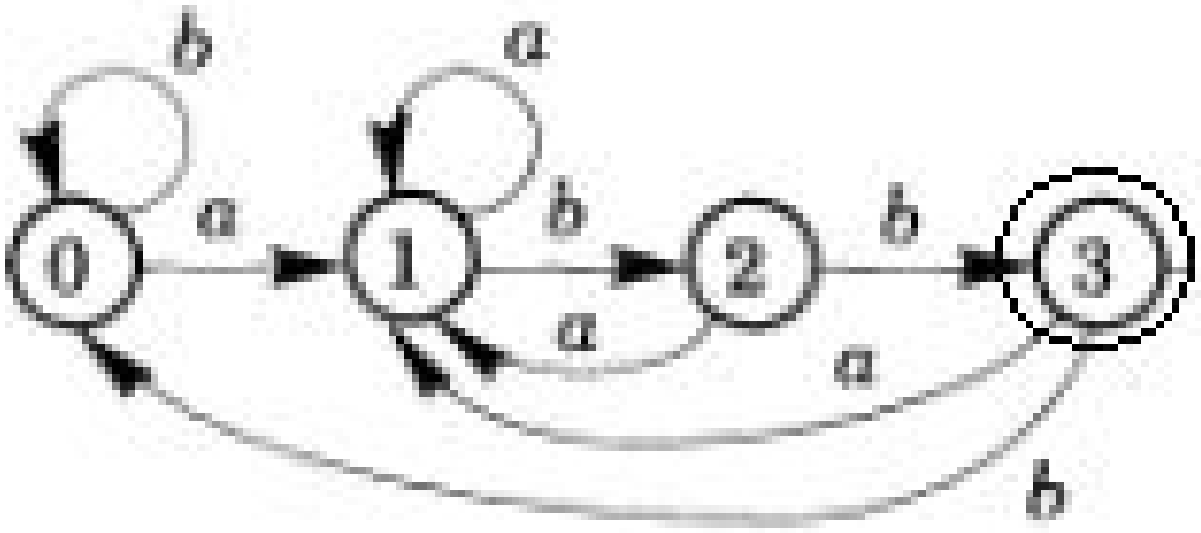
Finally, we apply rule 3 to ab to obtain the desired NFA for $a^* + ab$:



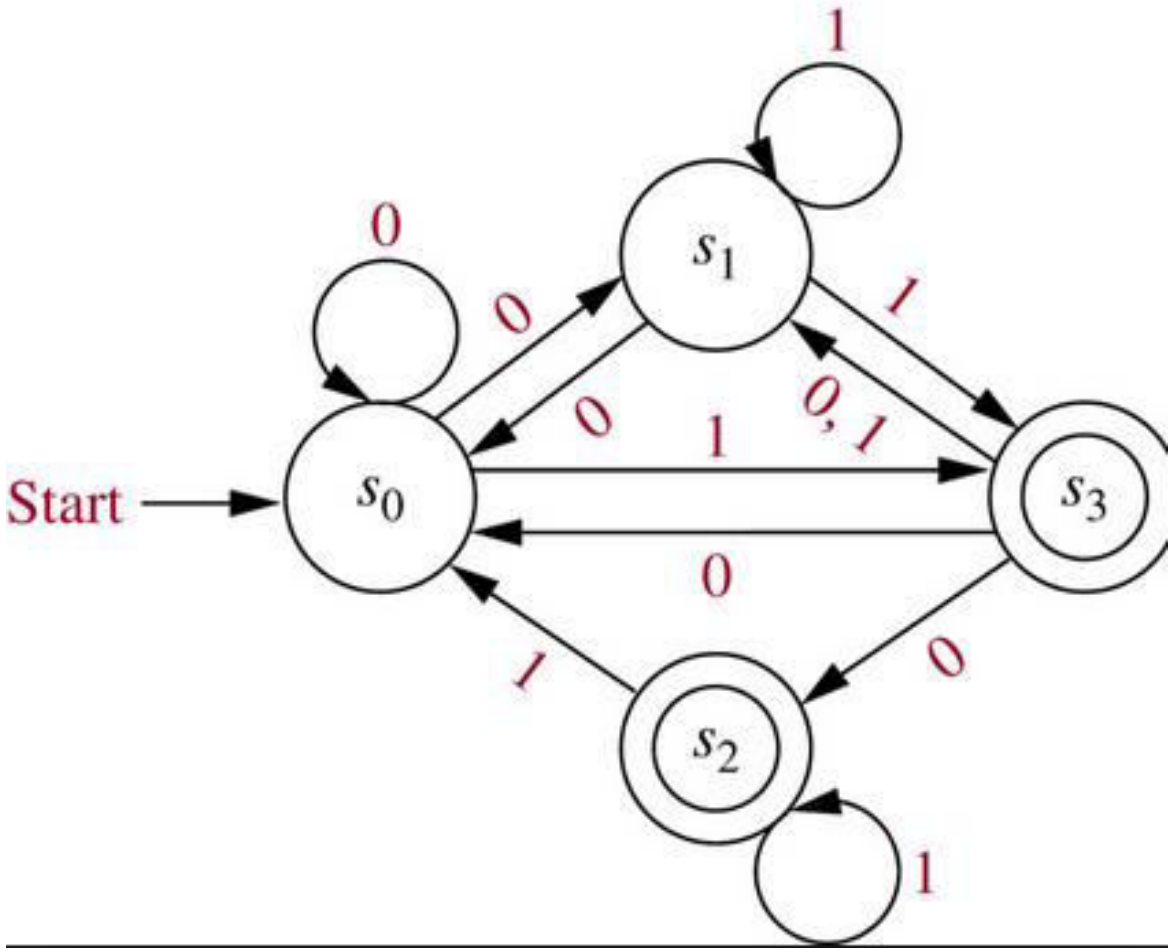
◆

Show that the following DFA is equivalent to R.E

(a+b)*abb

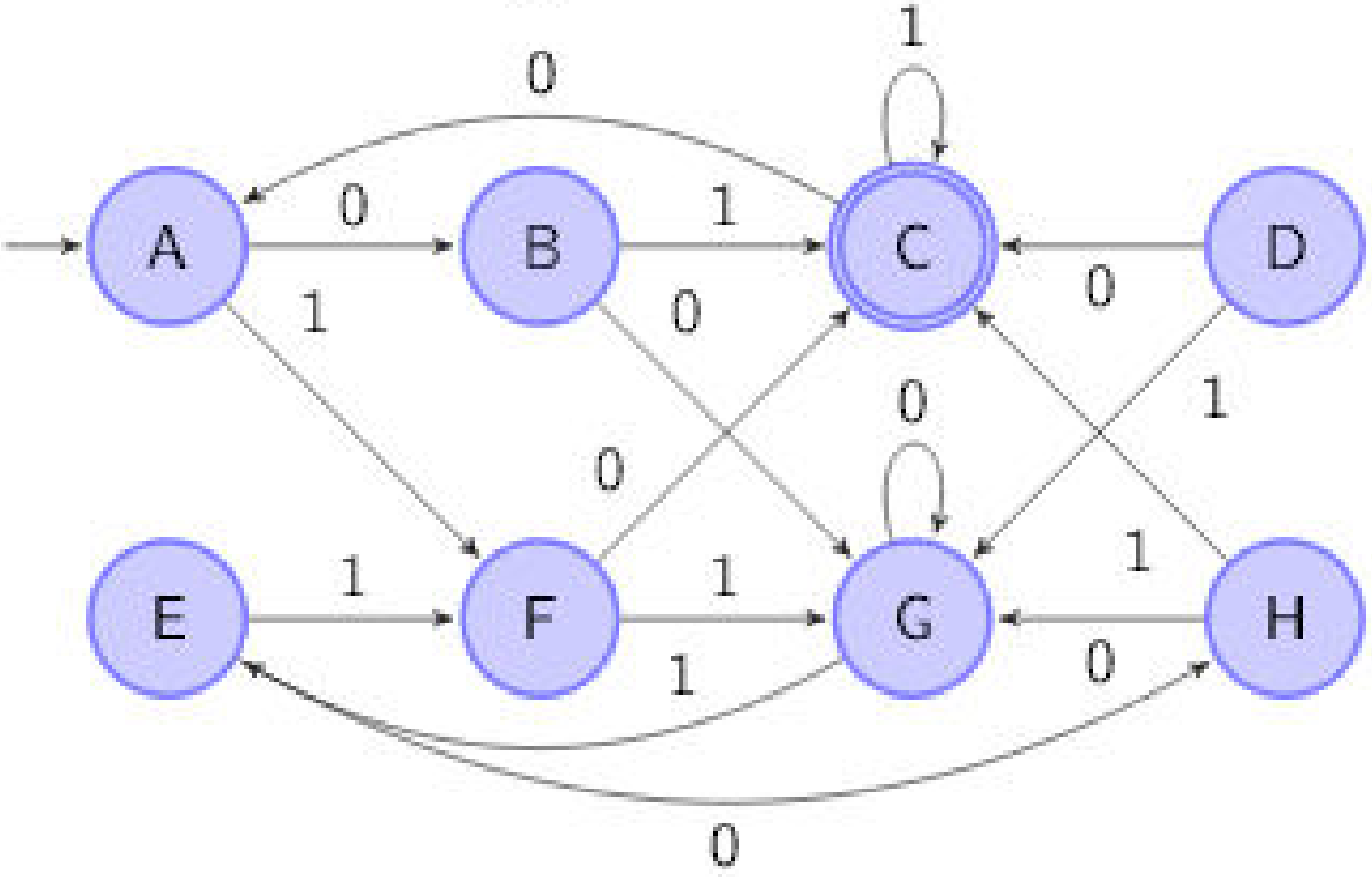


NFA to DFA Conversion

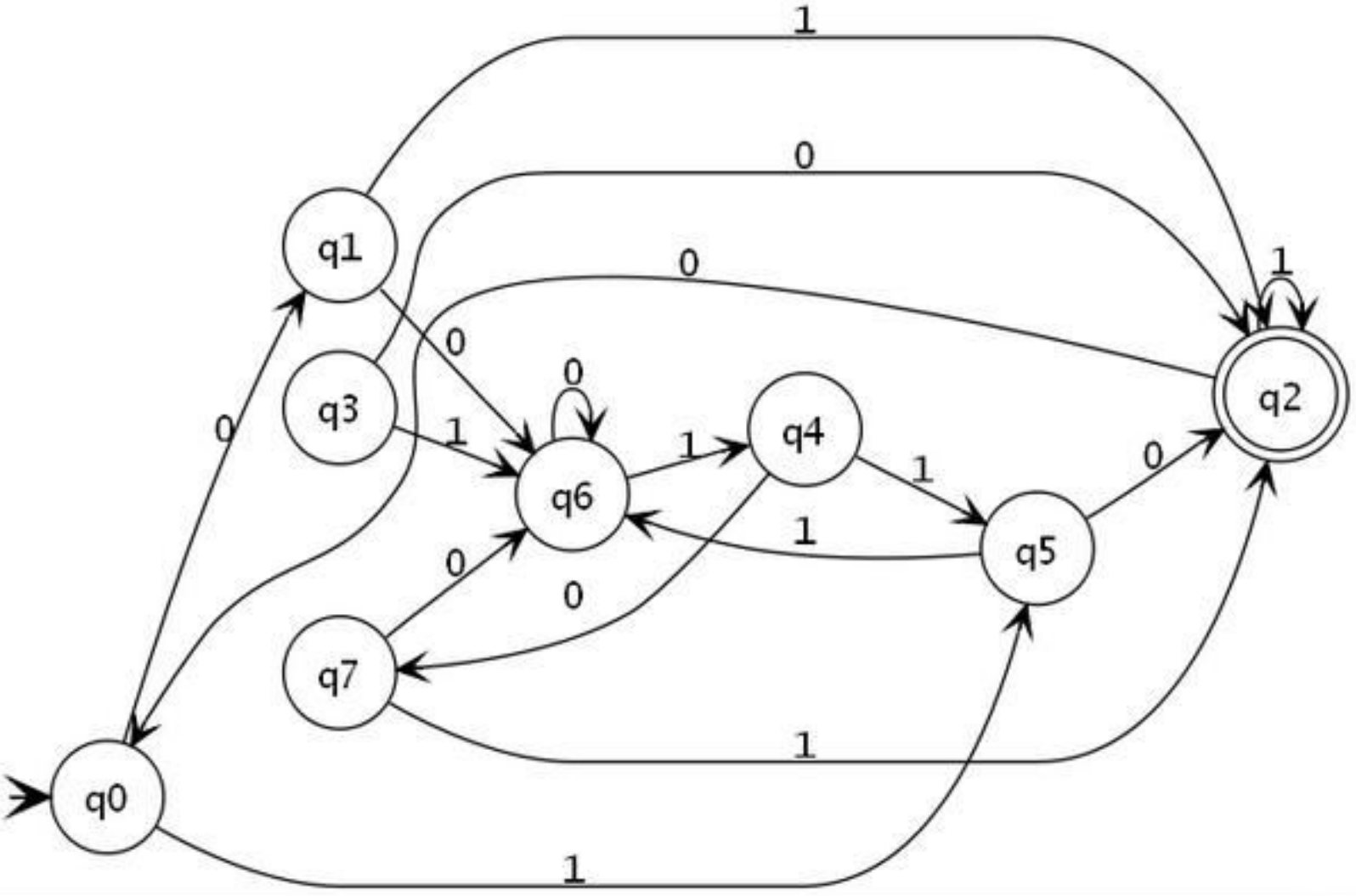


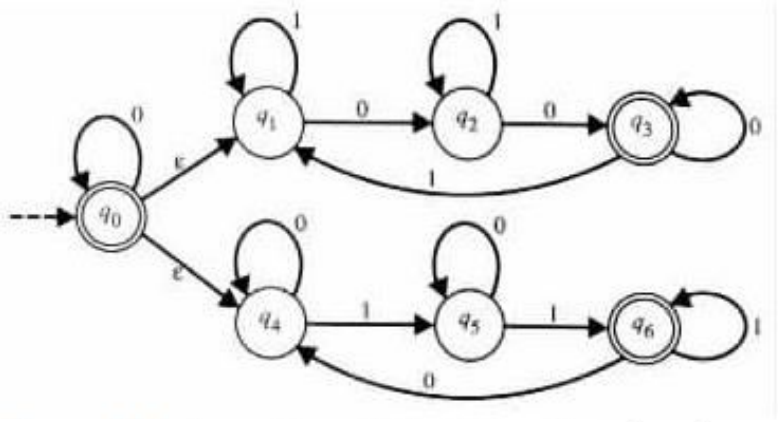
DFA Minimization

Minimize the following DFA:



DFA Minimization





automaton.

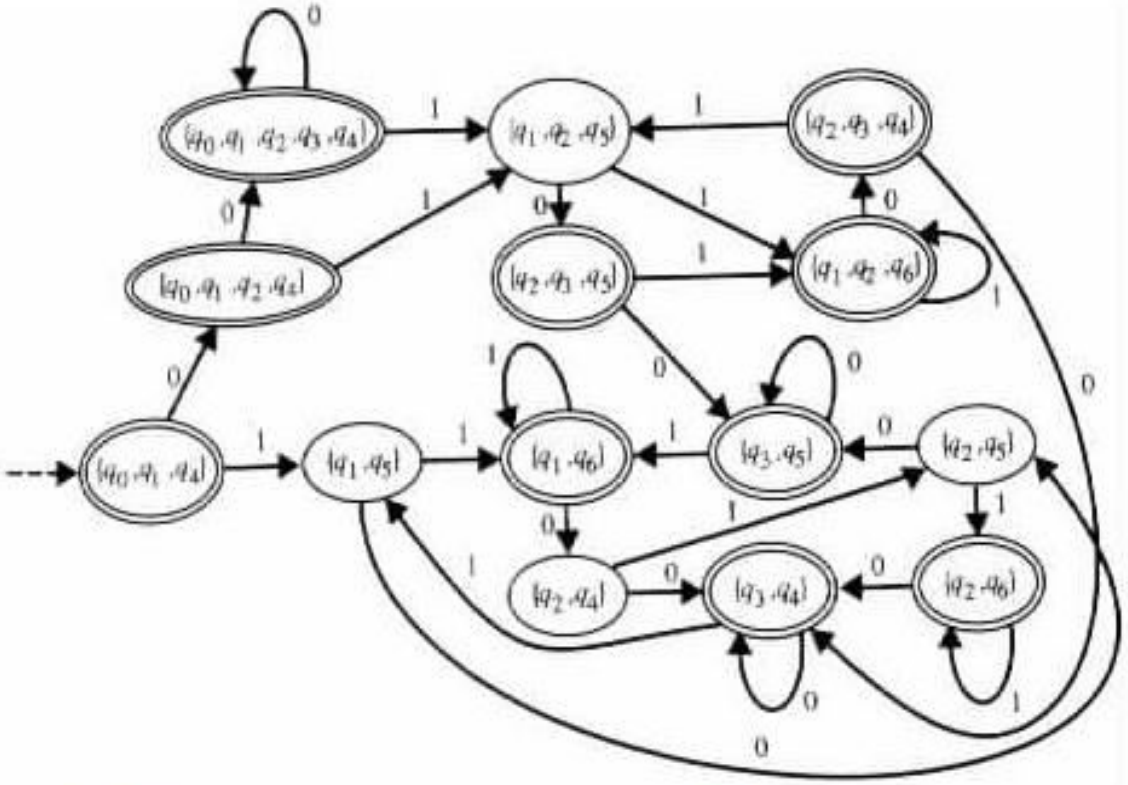


Figure 2.3.5 A transition diagram of an ϵ-free, deterministic finite-state automaton that is equivalent